A choice-free Lean formalization of Bishop–Cheng dominated convergence over presented reals

Toshihisa Urahigashi
Independent researcher
123026.urahigashi.toshihisa@gmail.com
ORCID 0009-0004-0460-6242

Submitted manuscript
Artifact version 0.7.2, 10.5281/zenodo.22309608

Abstract

Bishop and Cheng derive the dominated convergence theorem of their integral-first constructive measure theory through profiles: monotone functionals on families of continuous ramp functions, smooth, in the memoir's sense, at all but countably many levels. Choosing a level outside a countable exceptional set is exactly where a choice-free version of the theory is known to be obstructed—Spitters observed that the profile theorem implies the uncountability of the reals, which fails in an intuitionistic sheaf model.

This paper reports a Lean 4 formalization of that route: for an already integrable limit, convergence in measure together with domination on full sets that may depend on the index implies convergence of the integrals. The theorem is proved over *presented* reals—regular sequences of rationals with Bishop equality—and relative to an explicitly quantified integration-space interface, rather than for an abstract real-number object. The exceptional levels are enumerated and stepped around by Bishop's nested-interval construction, whose case split is decided from rational approximations and returned as data.

For the named public declarations the kernel reports an axiom footprint confined to `[propext, Quot.sound]`, with a static scan of the source closure, checksums, and a reproduced Rocq `Print Assumptions` audit of Séméria's earlier development in the Coq Repository at Nijmegen, over which no priority is claimed. Every integration space in Bishop and Cheng's sense, as transcribed, satisfies the interface used, and a finitely supported model, instantiated at the two booleans, gives both singletons positive measure: the axioms admit a measure not concentrated at a point.

Keywords: constructive measure theory; dominated convergence; Lean 4; proof assistants; presented reals; choice principles.

How to read this paper. Part I (Sections 1–6) is mathematical. It fixes the setting, states the definitions, states the theorem, and gives the proof, and it can be read without any knowledge of Lean or of proof assistants: no formal identifier occurs in a definition or in the statement of a theorem. Two places are exceptions to that, and both may be skipped on a first reading. Remark 2.2 names the modules and declarations that tie Definition 2.1 to the source's Definition 1.1, because the claim it makes is one a reader should be able to check rather than take on trust. And one construction is described below at a level of detail the rest of Part I does not use: the countable-avoidance step of Section 5 is described there down to the mechanism—where the cut points are placed and why the width contracts—but the recursion, the width estimate and the Cauchy data

are carried out in Appendix B.3, and a reader who wants that step at the level of a proof rather than a description should go there.

Throughout the paper, "the source" refers to Bishop and Cheng's memoir *Constructive Measure Theory* [2]; Bishop's *Foundations of Constructive Analysis* [1] is always cited by name.

Part II (Sections 7–11) concerns the formalization. It gives the shape of the public interface, the relation of the result to the known no-go theorems, and the comparison with earlier formalizations; it then reports the audit, the size of the development and how much of it the claims reach, and the limitations and reproduction procedure.

Appendix A tabulates the correspondence between the numbering of this paper and that of the source, and between the mathematical objects of Part I and the Lean declarations of Part II. Appendix B states, as ordinary mathematics and without Lean vocabulary, the constructions that the formalization had to supply beyond what the source provides, together with a classification of which parts are routine, which are known, and which were the actual difficulties. Part I suffices for the mathematical content; Appendix B is addressed to a reader who wants to know what the formalization concretely consisted of.

The dependencies at a glance. Each row is read left to right; the first four carry the mathematics, the fifth fixes what the theorem is stated about, and the sixth is the evidence. The rows are not a single chain: rows 1–3 and row 4 are two branches that meet, and the meeting is where the theorem is proved.

presented reals	→ cotransitivity as data	→ countable avoidance	Lem. 5.3
a profile	→ an explicit exception sequence	→ an apart admissible level	Thm 5.1, Lem. 5.3
an apart level	→ level sets, with domination	→ a uniform bound off a small set	Thm 5.2, Prop. 5.5
convergence in measure	→ an L^1 estimate	→ convergence of integrals	Thm 4.1
Definition 1.1	→ a hypothesis-free adapter	→ the working interface	Rem. 2.2
public declarations	→ their proof terms	→ `[propext, Quot.sound]`	§10.1

Rows 3 and 4 are the theorem, and they are separate because the hypotheses are: domination bounds the sequence, convergence in measure says it converges, and neither implies the other—the Lean statement takes them as two arguments, `hdom` and `hconv`. The L^1 estimate of row 4 is where they meet: it is obtained from convergence in measure together with the bound row 3 supplies. Rows 1 and 2 are the step at which a choice-free treatment is known to be obstructed, and row 2 consumes row 1: the level the profile is evaluated at is the one row 1 constructs. Row 5 is what makes the theorem apply to every integration space in the source's sense rather than to the interface alone.

Part I
The mathematics

1 Introduction

This paper reports that the Bishop–Cheng dominated convergence theorem can be carried out in Lean 4 without any appeal to Lean's `Classical.choice`, over reals that carry their own approximation data, and that the resulting development can be audited to that effect by a reader who runs a script. The theorem is Bishop and Cheng's and the proof is theirs; what is new is that the route survives being made choice-free at a place where a choice-free treatment was known to be

obstructed, and that the surviving development says so in a form a machine checks. A constructive machine-checked proof of the same theorem already exists in Séméria's CoRN development (Section 9); what this paper adds is a Lean 4 realization over a concrete presented-real type in which the obstructed step is discharged in the order theory of the reals, before any measure apparatus exists, together with an audit whose limits are stated as carefully as its results. Three things distinguish it from the closest prior formalization: domination is assumed only on a full set that may vary with the index; the countable-avoidance step is discharged in the order theory of the reals, before any measure apparatus exists; and the real numbers are internal to the artifact rather than an interface satisfied elsewhere, which is what the size of the development pays for. A fourth candidate—that propositions and data are kept apart by design—is *not* a difference, and Section 9.2 says why: the interface the prior work quantifies over already separates them. That section states the three precisely and says what is *not* different, which is the mathematics.

Dominated convergence is the basic passage from convergence of functions to convergence of their integrals. In the integral-first constructive measure theory of Bishop and Cheng, the proof is not simply a translation of the classical almost-everywhere argument. It proceeds through profiles of integrable functions, integrability of level sets away from an explicit exceptional sequence, and a uniform estimate on the complement of a suitable integrable set.

The reason is worth stating at once, because the whole paper turns on it. Classically, one proves that a monotone function $\mathbb{R} \to \mathbb{R}$ has at most countably many discontinuities, and one then picks a level t at which the distribution function $t \mapsto \mu(\{|h| > t\})$ is continuous. Both halves of that argument are problematic constructively. Bishop and Cheng replace the first half by the theory of profiles, which returns not an abstract countable set of bad levels but an *explicit sequence* of them. The second half—picking a level different from every member of an explicitly given sequence—then becomes a genuine construction rather than a triviality, and it is precisely the step at which choice-free constructive measure theory is known to encounter an obstruction: in an unrestricted constructive setting, the relevant profile theorem entails an uncountability principle that fails in an intuitionistic sheaf model [22].

This paper studies what can nevertheless be formalized when the real numbers are supplied with presentations. A presented real is not merely an element of an abstract ordered field. It carries rational approximation data and a modulus, and it returns as data the strict-order cotransitivity required by Bishop's nested-interval construction. The formalization uses that representation to build a point apart from every member of a given sequence, applies the construction to the exceptional levels of a profile, and then follows the Bishop–Cheng route to dominated convergence.

Here and below, *choice-free* has a deliberately audit-oriented meaning. It means that the named public Lean declarations, including their transitive proof-term dependencies, do not use `Classical.choice`, `Classical.choose`, `sorryAx`, or a comparable selector axiom. It does not mean that the theorems have no hypotheses, that every imported module is constructive, or that the profile theorem has been proved for every possible constructive real-number object. The reason for fixing the meaning this narrowly is stated in the literature: Berger and Seisenberger observe that "relativized choice—though trivially provable in constructive type theory—is in general not realizable" [17, §29.5]. Derivability inside a type theory therefore does not by itself guarantee realizability or the intended computational content, and a claim of this kind has to be attached to a named dependency closure and its axiom output (Section 8, Section 10.1).

The contributions of the artifact are as follows.

(1) **The theorem, and how the obstructed step is discharged.** Bishop and Cheng's Theorem 4.15 is formalized in Lean 4 by their own profile proof, over regular-sequence presented reals and against an explicitly quantified integration-space structure. The countable-avoidance

step is implemented by Bishop's nested-interval construction: convergence data for the endpoints, an apart limiting point, and the resulting smooth-level data are all constructed inside the audited dependency closure, so the step that the no-go results identify is discharged rather than assumed. The engine beneath the branch decisions is a machine-checked round trip, without choice, between the propositional and data forms of the strong order on presented reals (Appendices B.2 and B.12).

(2) **What the interface is, and what satisfies it.** A proof-relevant Type-valued route, automatic wrappers that synthesize smooth-level data, and a separately proved Prop-facing theorem are kept apart; the Type-valued route includes both global pointwise-majorant wrappers and a full-set variant in which the carrier and fullness proof for every index are explicit inputs, so the computational content stays visible without every caller supplying the low-level data. Concrete models show the interface is not vacuous: a finitely supported construction integrates by the unscaled sum over a list of points, so repetition gives a point an integer multiplicity, and clause (3) is discharged once a normalizing constant is supplied. On a general carrier the listed points need not be distinct—this interface carries no apartness structure on X—so non-degeneracy is settled at the closed instance on `Bool`, where both singletons are proved to carry positive measure. For a single point the continuity clause is the identity; for several, the point at which the series stays below the function has to be found, and it is found from the positivity witness of the strict inequality without any appeal to choice.

(3) **The evidence, and its boundary.** Reproducible Lean and Rocq audits, a pinned comparison with Séméria's earlier CoRN development, and explicit qualifications concerning predicativity, the integration-space interface, the concrete models, and priority. The size of the development is reported separately (Section 10.2).

Each principal claim is stated together with an explicit qualification. The following table pairs them; every row is developed in the section cited.

Claim	Direct evidence	Scope, and what is not claimed
A profile-route Bishop–Cheng dominated convergence theorem over presented reals (Theorem 4.1)	`bishop_cheng_dominated_convergence_propC` and the separately proved Type-valued route (Section 7)	relative to Definition 2.1 and to the presented reals of Section 2
The closest prior formalization is Séméria's CoRN development, which discharges the same obligation by a different route (Section 9)	the pinned Rocq `Print Assumptions` reproduction	no priority is claimed, in either direction
The working interface is derived hypothesis-free—all fourteen fields—from a clause-by-clause transcription of Definition 1.1 (Remark 2.2)	`toIntSpaceC` and `cutSmall_tendsto_of_src`; `logs/reading_layer_axioms.txt`	the memoir's own partial functions and its integral on L; the apartness structure on X is not represented
Full-set, index-dependent domination is the source's own §1 convention with the quantifiers displayed (Definition 3.4)	the memoir's §1 notational convention	not a weakening introduced by this paper
The named public declarations report `[propext, Quot.sound]` (Section 10.1)	kernel output, static source scan, checksums, shipped logs	no claim about ZF, about predicativity, or about mathlib as a whole
The countable-avoidance step is discharged constructively (Section 8)	the explicit exception sequence and the nested-interval construction (Appendix B)	the no-go results are not overturned; the claim is presented-reals-relative
The axioms admit a model not concentrated at a single point (Section 7)	`boolIndicator_integral_pos`: both singletons of the two-point model on `Bool` carry positive measure	the integral is the unscaled sum, so only integer multiplicities are realized; no model over a continuum is constructed

2 The setting

2.1 Presented reals

The scalar field is represented by presented real numbers: regular rational approximation sequences equipped with Bishop equality as the working equality relation [1, 3]. A presented real is thus a sequence (x_n) of rationals together with the regularity condition $|x_m - x_n| \leq m^{-1} + n^{-1}$, and two presented reals x, y are *equal* ($x \approx y$) when $|x_n - y_n| \leq 2/n$ for every n: equality of the numbers represented, weaker than literal agreement of the two sequences. A strict inequality $x < y$ is likewise always carried with a witness—an index and a gauge from which the difference stays positive—so that "how far apart, and from when" is part of the datum. Equality and convergence remain distinct: a sequence of presented reals converges to l when its terms approach l at every gauge with a stated rate, not because its terms are Bishop-equal to l.

Gauge and equality conventions. The reindexing between the two gauges is worth writing down, because both appear in this paper and comparisons made under one are not literal comparisons under the other. Given a sequence (x_n) regular for $1/n$, the sequence $x'_k := x_{2^k}$ is regular for 2^{-k}, since $|x'_j - x'_k| = |x_{2^j} - x_{2^k}| \leq 2^{-j} + 2^{-k}$; conversely a sequence regular for 2^{-k} yields one regular for $1/n$ by taking the term at the least k with $2^{-k} \leq n^{-1}$. The two forms therefore present the same reals, but a statement about x_n and one about x'_k are statements about different terms,

and the appendix means the dyadic ones throughout. Equality is taken in the implementation in the tail form $\forall k\, \exists N\, \forall n \geq N \colon |x_n - y_n| \leq 2^{-k}$—the form stated in Definition B.1, and the relation `relEventually` in the code. For regular sequences this is equivalent to the displayed condition—one direction is immediate, and the other follows from regularity, since $|x_n - y_n| \leq 2 \cdot 2^{-n} + 2 \cdot 2^{-m} + 2^{-k}$ for every $m \geq N$—but the development adopts the tail form as its definition rather than deriving the equivalence internally.

The development does not identify Bishop-equal approximation sequences: the carrier is the type of regular sequences, not a quotient of it, and every statement is a relation between sequences rather than between equivalence classes. This is a transcription of Bishop's own position in the *Foundations* rather than an implementation convenience: the notes to that chapter state that a real number *is* the regular sequence, and not an equivalence class of regular sequences, arguing that the equivalence-class reading would be inaccurate or meaningless. Analytic statements are formulated so that they respect the Bishop equivalence relation.

Two consequences matter later. First, every real carries its own modulus by construction. Second, given a strict inequality $a < b$ and a third presented real c, the required cotransitivity branch can be *decided from rational approximation data and returned as data*: from sufficiently good rational approximations one can tell which of $a < c$, $c < b$ holds, and one may return that information rather than only the proposition "$a < c$ or $c < b$". It is this data form of cotransitivity that makes Bishop's nested-interval construction (Lemma 5.3 below) a construction rather than an appeal to a choice principle; the mechanism by which the branch is decided is written out in Appendix B.2.

That second consequence is Richman's construction, quoted here because it is precisely the step at which a classical argument divides into cases: "if $a < b$ are rational numbers, then we can show that either $x < b$ or $x > a$. Indeed let x' be a rational number within $(b - a)/3$ of x. If $x' \leq (b + a)/2$, then $x < b$; while if $x' \geq (b + a)/2$, then $x > a$" [7]. The branch is settled by one rational comparison at a computable accuracy, and that outcome is the datum returned by the order layer (Appendix B.2). Richman also states the moral this development follows throughout: a case split "only says that the *argument* is discontinuous as a function of the data, not that this discontinuity is an essential feature of the theorem" [7].

Supplying such a datum is the standard device of the subject rather than a local expedient—"we can constructivize a classical notion by supplying additional information", and moduli witness not only Cauchyness but uniform continuity, uniform convergence, differentiability and monotonicity as well [15, §27.2.1]—and it is not free. Bickford states the price exactly: a theorem proved under a uniform-continuity hypothesis "requires a modulus of continuity as an input", whereas a hypothesis with no constructive content needs none [16, §28.1]. The Type-valued route of Section 7 makes that payment explicit; the Prop-facing surface hides it again from callers who do not need it.

2.2 Integration spaces

The measure-theoretic layer follows a Bishop–Cheng-style route [2]. We work with *partial* functions: an element of the ambient function class is a pair consisting of a map into the presented reals and a domain, and the sum of two partial functions is defined on the intersection of their domains.

Both features are the source's own choices. A measure-first constructive development is possible—it is carried out in the memoir itself—but the earlier book "adopted the much more convenient Daniell-integral approach", postulating L and E side by side and building measurability and the measure from them [9, §13.1.2]. Admitting functions defined only almost everywhere is equally deliberate, and Chan names its cost: it "does add the small task of checking, whenever we create an integrable function from prior ones, that the condition of being defined a.e. is duly

inherited" [9, §13.1.2]. That task is a substantial part of the partial-function bookkeeping below.

All results below are proved relative to an explicitly quantified structure, an *integration space* in the following working sense.

Definition 2.1 (Integration space, working form; cf. the source's Definition 1.1)**.** An integration space is a triple (X, L, I) consisting of a type X, a class L of partial functions on X with values in the presented reals, and a functional $I : L \to \mathbb{R}$, such that:

(A1) L and I respect Bishop equality of partial functions;

(A2) L is closed under addition, multiplication by a scalar, and absolute value—$f + g$, αf, and $|f|$ lie in L whenever $f, g \in L$ and $\alpha \in \mathbb{R}$—and under truncation at a constant carrying a positivity witness: $\min\{f, \alpha\}$ lies in L whenever $f \in L$ and $0 < \alpha$ is held as data. Truncation at 0 is not a field but a derived lemma: $\min\{f, 0\} = \frac{1}{2}(f - |f|)$, the negated negative part of f, follows from linear combination, absolute value, and respect for equality (Appendix B.6);

(A3) I is linear: $I(f + g) = I(f) + I(g)$ and $I(\alpha f) = \alpha I(f)$;

(A4) I is positive: $I(f) \geq 0$ whenever $f \in L$ is pointwise nonnegative on its domain;

(A5) the two truncation limits hold for every $f \in L$:

$$I(\min\{f, n\}) \longrightarrow I(f), \qquad I(\min\{|f|, n^{-1}\}) \longrightarrow 0 \qquad (n \geq 1);$$

(the implementation phrases the small truncation with the dyadic gauge 2^{-n}; the difference of index domains is absorbed by the gauge translation of Section 10.4);

(A6) *(constructive continuity)* if $f \in L$, if (f_n) is a sequence of pointwise nonnegative members of L, if $\sum_n I(f_n)$ converges, and if $\sum_n I(f_n) < I(f)$, then there is a point x lying in the domain of f and of every f_n such that $\sum_n f_n(x)$ converges and $\sum_n f_n(x) < f(x)$.

Axiom (A6) is Bishop and Cheng's clause (2), the constructive analogue of the continuity condition for a Daniell integral, stated in the positive form which returns a point at which a series of nonnegative functions stays below a given function. The two limits in (A5) are the same pair of limits that Spitters isolates in his definition of an integration f-algebra [22].

Remark 2.2 (Fidelity: a clause-level strict weakening of Definition 1.1). Definition 2.1 is *not* a verbatim rendering of Bishop and Cheng's Definition 1.1 [2]. It is *strictly weaker*, at the level of the displayed clauses, than Definition 1.1 as encoded in this development—the encoding, and the two scope qualifications it carries, are made precise below. Here and below, "clauses (1)–(4)" refer to the four clauses of the source's Definition 1.1—(1) closure under linear combination, absolute value, and $\min\{f, 1\}$, together with linearity of I; (2) series continuity, yielding a point; (3) normalization $I(p) = 1$; (4) the two truncation limits—while (A1)–(A6) refer to Definition 2.1.

In two respects Definition 2.1 is weaker. The source requires in addition that X be inhabited and that there exist $p \in L$ with $I(p) = 1$; neither requirement appears above. Definition 2.1 is therefore satisfied by a degenerate zero-integral model, shipped with the artifact (Section 7.4). A second, *normalized* example is point evaluation at a fixed point (`Mathdemo.DiracIntegrationSpace`), for which the constant function 1 lies in L with $I(1) = 1$, so that clause (3) holds. Both source-level limits of clause (4) are proved, so the model is unconditional. For point evaluation they reduce to statements about the truncations of a single real: $\min\{c, n\} \to c$ holds with a constant modulus once n passes an Archimedean bound for c, and $\min\{|c|, 1/(n+1)\} \to 0$ holds because the truncation is squeezed between 0 and $1/(n + 1)$; the module also proves the auxiliary dyadic limit

$\min\{|c|, 2^{-n}\} \to 0$. The Archimedean bound is the witness $2^{\mathrm{mulArchBound}(c)}$ of `exists_nat_geC` written out explicitly, since that lemma is a `Prop`-valued existential and cannot supply a modulus, which is data. Since the adapter is hypothesis-free, this clause-by-clause encoded model yields a model of Definition 2.1 with no further hypothesis.

The closure clause of (A2) is stated for constants carrying a positivity witness, and the restriction is not cosmetic. Clause (1) asks only that $\min\{f, 1\} \in L$, and the constants recoverable from it are positive: the source's own remark derives $\min\{f, n\} = n \cdot \min\{n^{-1}f, 1\}$. Closure under truncation at an *arbitrary* constant is not available, and the obstruction is visible in Bishop and Cheng's own canonical model $(X, C(X), \mu)$ of their Definition 7.1—X locally compact, $C(X)$ the continuous functions of compact support, μ a positive measure: one has $\min\{f, \alpha\} = \alpha$ off the support of f, so for $\alpha < 0$ and non-compact X the truncation leaves $C(X)$ (Appendix B.6). An interface demanding truncation at arbitrary constants would therefore not be implied by Definition 1.1, and the two would not be ordered by logical strength.

Restricting the clause to $\alpha \geq 0$ does not by itself repair this. The recovery route the source itself supplies is $\min\{f, n\} = n \cdot \min\{n^{-1}f, 1\}$, which requires α^{-1} and hence a witness that α is apart from 0; the case $\alpha = 0$ is recovered separately, since $\min\{f, 0\} = (f - |f|)/2$ follows from clause (1) and closure under $|\cdot|$ by a lattice identity that needs no case analysis (Appendix B.6). Constructively one cannot decide between $\alpha = 0$ and $\alpha > 0$, and we have found no uniform construction from $\alpha \geq 0$—that is, from $\neg(\alpha < 0)$—alone. The source-derivable form, sufficient for every use in the present development, is therefore truncation at constants *carrying a positivity witness*, or at 0: exactly the two kinds of constant the development truncates at. The closure clause of Definition 2.1 is accordingly the field `cutPos_mem`, truncation at a constant accompanied by positivity data (`PosEventuallyData`); truncation at 0 is the derived lemma `cutZero_mem` rather than a field; and a witness type `CutConstWitnessC`, with a positive branch and a zero branch, wires the evidence through the development. Every use site goes through with the canonical witness; a survey of all use sites found none truncating at a general constant. We emphasize that we have not proved that $\alpha \geq 0$ is insufficient; the claim is only that the source's route needs a witness and that no other constructive route is evident to us.

The adapter is hypothesis-free. The module `Mathdemo.SourceIntegrationSpaceDef11` contains a clause-by-clause transcription `IntegrationSpaceDef11` of the displayed Definition 1.1 together with an adapter `toIntSpaceC` into Definition 2.1: *all fourteen fields of the working interface are machine-derived, without extra hypotheses, from the clause-by-clause transcription.* Thus Definition 1.1 supplies every field required by Definition 2.1. (No gauge convention is carried across: the transcription states the small-cut limit of clause (4) at the source's own gauge n^{-1}, built as a presented real from a positivity witness and with no rational inverse assumed, and the passage to the dyadic gauge 2^{-n} used by Definition 2.1 is itself machine-checked as `IntegrationSpaceDef11.cutSmall_tendsto_of_src`, the reverse gauge comparison `srcGauge_le_halfPow` is used by the point-evaluation model.) Two scope qualifications bound this claim. First, the ambient function class is rendered as the memoir renders it: an element of `BFunC` is a domain together with a map defined *only on that domain*, and the integral is a map defined *only on* L, so that the transcription asks for no datum the source does not supply. What is still not represented is the memoir's apartness structure on X and the strong extensionality it builds into $F(X)$: those are omissions, not additional demands. Second, at the level of the four displayed clauses the only remaining differences are the omissions of inhabitedness and normalization, both pure weakenings. Consequently the current Definition 2.1 is strictly weaker than the transcribed Definition 1.1, and *every integration space in the sense of Definition 1.1 satisfies Definition 2.1 and the theorems of this paper apply to it.* The clause-level weakening is proper: the zero-integral model on `PUnit` satisfies Definition 2.1 but, having $I \equiv 0$, cannot satisfy the normalization clause $I(p) = 1$ of Definition 1.1, since 1 is

apart from 0 in the presented reals. The intermediate steps follow the source: clause (1), whose linear-combination part is the assertion $\alpha f + \beta g \in L$ together with $I(\alpha f + \beta g) = \alpha I(f) + \beta I(g)$, yields closure under addition and under scalar multiplication and the additivity and homogeneity of I (each through the equality of $F(X)$); clause (2) yields Corollary 1.3 at $k = 1$ (`pos_witness`) and from it `I_nonneg`; and clause (1)'s $\min\{\cdot, 1\}$ yields `cutPos_mem` by the same algebra as the source's positive-integer remark, generalized to constants carrying positivity data. Every one of these declarations reports `[propext,Quot.sound]`.

Two further observations complete the inventory. First, Definition 2.1 carries `I_nonneg` as a field, but need not: it is derivable from clause (2), by the derivation the source performs in Lemma 1.2, Corollary 1.3, and the sentence following Corollary 1.3, and that derivation is machine-checked (Appendix B.6). Second, and this is *not* a difference: clause (2) is rendered with a point-returning structure rather than a `Prop`-valued $\exists$, and the limits of clause (4) with an explicit modulus rather than an ε-δ statement. That is faithfulness rather than strengthening, since Bishop's $\exists$ asserts under the BHK reading that the object can be constructed, and a Bishop limit always carries a modulus; the `Prop`-valued reading would be the weaker one. As a Lean interface, however, the data-carrying rendering is a data refinement of the corresponding `Prop`-valued statement rather than an equivalent restatement (Section 7.2).

The full field list is machine-readable in the artifact, so a reader can check exactly which hypotheses the theorems consume.

3 Integrable sets, full sets, profiles

This section collects the notions used in the statement of the theorem. All of them are Bishop and Cheng's [2]; we recall them because the theorem cannot be read without them.

Definition 3.1 (Full set; the source's Definition 1.9)**.** A subset $A \subseteq X$ is a *full set* if it contains a countable intersection of domains of integrable functions.

Definition 3.2 (Complemented set; the source's Definitions 2.1, 2.2(d)(e), 2.3)**.** A *complemented set* is an ordered pair $A \equiv (A^1, A^2)$ of subsets of X such that $x \in A^1$ and $y \in A^2$ imply $x \neq y$. We write $x \in A$ for $x \in A^1$ and $x \in -A$ for $x \in A^2$, and put $-A \equiv (A^2, A^1)$ and $A - B \equiv A \wedge (-B)$. A complemented set is *integrable* when its characteristic function is, and its *measure* $\mu(A)$ is the integral of that characteristic function.

Complemented sets carry their own witnesses of non-membership; this is what replaces the classical complement, for which one has no constructive access.

Definition 3.3 (Convergence in measure; the source's Definition 4.11)**.** A sequence (f_n) of measurable functions *converges in measure* to a function f defined on a full set $D(f)$ if for every integrable set A and every $\varepsilon > 0$ there is N such that for every $n \geq N$ there is an integrable set B with

$$B^1 \subseteq A^1 \cap D(f) \cap D(f_n), \qquad \mu(A - B) < \varepsilon, \qquad |f(x) - f_n(x)| < \varepsilon \ \text{ for all } x \in B.$$

Note that the good set B, its integrability, the small-complement estimate, and the pointwise closeness all appear in the statement; nothing is left to an external library convention.

Definition 3.4 (Domination on full sets; the source's §1 convention with the quantifiers displayed)**.** In §1 of the memoir, Bishop and Cheng adopt the notational convention that, for arbitrary $f, g \in F(X)$, the inequality $f \leq g$ means that $f \leq g$ holds *on some full set*. The domination hypothesis $|f_n| \leq g$ of their Theorem 4.15 is therefore already a full-set inequality on the source's own reading,

and since the inequalities for different n are separate statements, the full set may depend on n. The present definition displays that quantification. A sequence (f_n) of integrable functions is *dominated on full sets* by an integrable g if for every n there is a full set F_n such that $|f_n(x)| \le g(x)$ for every $x \in F_n$ at which the relevant values are defined. The full set is allowed to depend on n.

Finally, the notion at the heart of the proof.

Definition 3.5 (Profile; the source's Definition 3.1)**.** Let $[a, b] \subset \mathbb{R}$ be a compact proper interval and let $C[a, b]$ be the continuous functions on it. Let $F \subseteq C[a, b]$ satisfy

(a) $0 \le f \le 1$ for every $f \in F$;

(b) the constant functions 0 and 1 belong to F;

(c) if $a \le u < v \le b$ then some $f \in F$ satisfies $f(t) = 0$ for $t \le u$ and $f(t) = 1$ for $t \ge v$.

Let $\lambda : F \to \mathbb{R}$ be non-decreasing, i.e. $\lambda(f) \le \lambda(g)$ whenever $f \le g$ pointwise on $[a, b]$. Then (F, λ) is a *profile* on $[a, b]$.

The example to keep in mind is $\lambda(f) = \int_a^b f(x)\,dx$; this sentence is the source's own. A profile is thus a monotone functional on a supply of ramp functions, and condition (c) says that the supply is rich enough to separate any two levels. (The formalization states the definition in a coded form in which condition (c) returns the separating function; Appendix B.4. The coded structure records (a)–(c) and the monotonicity of λ, and carries *no* continuity clause: where the source takes $F \subseteq C[a, b]$, the formalized profile requires of its members only the displayed conditions. This weakens the hypothesis, so the formalized Theorems 5.1 and 5.2 are correspondingly more general than the source's. The structure has no continuity field, so nothing downstream can depend on continuity.)

Definition 3.6 (Smooth level; introduced in the source inside the text of its Theorem 3.5)**.** A point $t \in [a, b]$ is *smooth* for the profile (F, λ) if there is a number $\overline{\lambda}(t)$ such that for every $\varepsilon > 0$ there is $\delta > 0$ with the property that

$$|\lambda(f) - \overline{\lambda}(t)| < \varepsilon$$

for every $f \in F$ with $f(x) = 1$ for all $x \ge t + \delta$ and $f(x) = 0$ for all $x \le t - \delta$.

For the model example $\lambda(f) = \int f$, smoothness at t says that ramps which turn on inside a small window around t all have nearly the same integral: the profile does not jump at t. Smoothness is therefore the constructive substitute for continuity of a monotone function at t, and $\overline{\lambda}(t)$ is the common value.

4 The theorem

Theorem 4.1 (Dominated convergence; the source's Theorem 4.15)**.** *Let (X, L, I) be an integration space in the sense of Definition 2.1. Let f and f_n $(n \in \mathbb{N})$ be integrable functions on X, and assume:*

(i) $f_n \to f$ in measure, in the sense of Definition 3.3; and

(ii) there is an integrable g dominating (f_n) on full sets, in the sense of Definition 3.4.

Then

$$\forall \varepsilon > 0\ \exists N\ \forall n \ge N, \qquad |I(f_n) - I(f)| < \varepsilon.$$

On numbering: this paper numbers items by section, so the statement above is Theorem 4.1, and it is the same assertion as Theorem 4.15 of the memoir (the differences in how the hypotheses are phrased are discussed in Remark 4.3 and Section 6); that both numbers happen to begin with "4" carries no meaning. Appendix A.1 tabulates the correspondence between the numbering of this paper and that of the source. In the sequel, "Theorem 4.1" refers to this paper's numbering and "Theorem 4.15" to the source's.

Remark 4.2 (Scope visible in the statement). The limit f is itself assumed integrable; the theorem does not derive the integrability of an otherwise merely measurable limit. The result is also relative to the structure (X, L, I) and to presented reals. These are mathematical hypotheses of the theorem, not axioms hidden from Lean's dependency report.

Remark 4.3 (The mode of the hypotheses). Two features of the hypotheses are worth flagging for comparison with other formulations. First, the mode of convergence assumed is convergence in measure, not pointwise convergence. Second, the domination hypothesis is a full-set hypothesis, and the full set is allowed to vary with the index n; a single global pointwise bound is not required. The second feature is *not* a weakening introduced by this paper: it renders explicit what the source's §1 convention already means under Theorem 4.15 (Definition 3.4). Likewise, the difference between the error majorant $H = |g| + |f|$ used in the proof below and the source's $2g$ is not a difference of hypotheses but a difference of proof route: the memoir's displayed estimate uses $2g$, which presupposes an inherited full-set bound $|f| \le g$; that inheritance is classically standard but is neither a stated hypothesis of Theorem 4.15 nor established in the displayed proof, and the verified development does not derive it from convergence in measure and the bounds on the f_n, so it uses the immediately available majorant instead (Section 6). Section 9.1 compares the hypothesis with the corresponding CoRN hypothesis.

5 Profiles, level sets, and countable avoidance

The proof of Theorem 4.1 rests on three results. The first two are Bishop and Cheng's; the third is the constructive content of the step that the classical proof takes for granted.

Theorem 5.1 (Smoothness away from an explicit sequence; the source's Theorem 3.5)**.** *Let (F, λ) be a profile on $[a, b]$. Then all but countably many points of $[a, b]$ are smooth in the sense of Definition 3.6.*

The statement is the source's, and "all but countably many" is read here in its constructive sense: what is produced is an explicit sequence T, and every point *apart from* every term of T is smooth. That is the form the development proves and the form the rest of this paper uses, so it is worth having in front of the reader—it is Theorem B.11: one constructs an explicit sequence $T = (T_n)$ such that every t *apart from* every T_n is a smooth level. Apartness, not mere difference, is what the sequel consumes, and it is what Lemma 5.3 delivers.

The proof is what matters here. Bishop and Cheng choose an increasing sequence $\{n(k)\}$ of positive integers with $(n(k) + 1)k^{-1} > \lambda(1) - \lambda(0)$, collect the finitely many points $\{t_1^k, \ldots, t_{n(k)}^k\}$ excluded at stage k, and set $T = \{t_j^k\}$. Every t *apart* from every member of T—that is, satisfying $0 < |t - T_n|$ for every enumerated term, which is what the formalized statement requires and is strictly stronger than $t \neq T_n$—is then shown to be smooth, with $\overline{\lambda}(t)$ constructed explicitly. Thus the conclusion is not that a certain set "is countable" with an existentially hidden enumeration: the exceptional levels arrive as an *explicit sequence*, constructed by the source's own proof. The formalization's contribution at this point is packaging rather than strengthening: it flattens the

proof-internal data into a first-class $\mathbb{N}$-indexed sequence that later constructions consume directly (Appendix B.5).

Theorem 5.2 (Integrability of level sets; the source's Theorem 3.6)**.** *Let (X, L, I) be an integration space and h an integrable function. Then for all but countably many $t > 0$ the complemented sets*

$$A \equiv (\{x : h(x) \geq t\},\ \{x : h(x) < t\}), \qquad B \equiv (\{x : h(x) > t\},\ \{x : h(x) \leq t\})$$

are integrable and have the same measure.

The bridge between the two theorems is the profile built from h. For $0 < u < v$ let $f_{u,v}$ be the ramp which is 0 below u, rises linearly on $[u, v]$, and is 1 above v, and set $\lambda_0(f_{u,v}) = I(f_{u,v} \circ h)$; the composite is integrable because

$$f_{u,v} \circ h \ = \ \min\{(v-u)^{-1}(h - \min\{h, u\}),\, 1\}.$$

Restricting to $[a, b]$ gives a profile in the sense of Definition 3.5, and $\overline{\lambda}(t)$ is the common measure of A and B. So the distribution function of h appears as a profile, its jump levels appear as the explicit sequence T, and integrability of a level set is available exactly at levels away from T. (Constructively, the two pairs A and B are built independently: since $\leq$ is the negation of $<$, neither pair is the complement of the other, and the classical one-line passage from A to B is not available; Appendix B.8.)

This is where a constructive proof must do work that the classical proof does not.

Lemma 5.3 (Countable avoidance by nested intervals; Bishop [1, Ch. 2, Theorem 1])**.** *Let $a < b$ be presented reals and let (q_n) be a sequence of presented reals. Then one can construct t with $a < t < b$ and t apart from q_n for every n.*

The construction is the familiar one: run a nested sequence of closed intervals shrinking inside (a, b), at stage n choosing a subinterval that avoids q_n, and take the limit. The step "choose a subinterval avoiding q_n" is a case distinction on the position of q_n relative to two candidate subintervals, and it is exactly here that presented reals are used: the cotransitivity split is decided from rational approximations and returned as data, so the sequence of intervals is a construction and not a sequence of arbitrary choices. The limit exists because the left endpoints form a Cauchy sequence with an explicit modulus. The step that a classical proof would take by bisection cannot be taken here: the position of q_n relative to a *single* point is not decidable, and the Brouwerian example that follows Theorem 1 of Chapter 2 of the *Foundations* exhibits a real for which even $x \geq 0$ or $x \leq 0$ is not provable. Bishop's own proof of that theorem does not bisect. It compares the intruding real against a *separated pair*—at stage n, either $a_n > x_{n-1}$ or $a_n < y_{n-1}$, the branch being furnished by the corollary to Proposition 7 of the same chapter, that $y < z$ implies $x < z$ or $x > y$—and keeps a subinterval clear of it, maintaining the invariant that $x_n > a_n$ or $y_n < a_n$ while the width falls below n^{-1}. What the formalization supplies is a determinization of that step: the cut points are fixed *in advance* at a margin $\sigma(w) = \frac{1}{4}w$ of the current width, so that the comparison is made against a pair whose separation is known before q_n is looked at, the outer quarter on the far side is retained and the middle half discarded, and the branch is returned as data rather than asserted. Each step then reduces the width to $\frac{1}{4}$ of its previous value, with an explicit modulus. The construction, with the interval data carried at each stage, is Appendix B.3; Appendix B.11 lists it among the difficulties of the *formalization*, which are not claims of mathematical novelty.

Remark 5.4 (The conclusion in data form). Theorem 4.1 is stated above in the $\forall\varepsilon\,\exists N$ form, which is the form the source states and the form the Prop-facing declaration carries. The development

also proves a data-valued route with the same hypotheses supplied as data: from convergence-in-measure data, domination data on full sets, and the integrability representations, one *constructs* a modulus $k \mapsto N(k)$ with

$$|I(f_n) - I(f)| \;\le\; 2^{-(k+1)} \qquad \text{for all } n \ge N(k).$$

The rate is part of the output, not merely asserted to exist. Section 7.2 describes the two routes and the hypotheses each of them takes.

Applying Lemma 5.3 to the exceptional sequence T of Theorem 5.1 constructs an admissible level, and hence, by Theorem 5.2, integrable level sets at that level. The following consequence is the form in which the profile theory enters the convergence argument.

Proposition 5.5 (Uniform complement estimate)**.** *Let H be a nonnegative integrable function and let (e_n) be a sequence of nonnegative integrable functions such that for each n there is a full set on which $e_n \le H$ (in the application to Theorem 4.1, $e_n = |f_n - f|$ and $H = |g| + |f|$). Then for every $\varepsilon > 0$ one can construct an integrable set A, an index bound N, and a measure threshold $\delta > 0$ such that for every $n \ge N$ and every integrable complemented set C,*

$$\mu(A - C) < \delta \quad\Longrightarrow\quad I\big(e_n - \chi_C\, e_n\big) < \varepsilon.$$

Proposition 5.5 corresponds to the hypotheses of the source's Lemma 4.14 together with the construction inside its Theorem 4.13, under the equivalent reparameterization $B := -C$: the source's smallness condition $\mu(A \wedge B) < \delta$ is the condition $\mu(A - C) < \delta$ above, since $A - C = A \wedge (-C)$. It is what makes the "outside a good set" half of the classical proof work without an almost-everywhere argument: the smallness of the integral off A is obtained from the measure of a level set of H at an admissible level, and admissibility is supplied by Lemma 5.3. The triple (A, N, δ) is returned as data because the sequel consumes it (Appendix B.10).

6 Proof of Theorem 4.1

The proof follows the decomposition below. The arrows indicate constructed data or a theorem application, rather than an informal implication between black-box library results.

profile partition data and an explicit exception sequence
$\Downarrow$ presented-real countable avoidance
an apart threshold and integrable level sets
$\Downarrow$ profile estimates
a uniform small-complement integral bound

convergence in measure + full-set domination
$\Downarrow$ good-set/complement decomposition
$I(|f_n - f|) \longrightarrow 0$
$\Downarrow$ $|I(f_n) - I(f)| \le I(|f_n - f|)$
$I(f_n) \longrightarrow I(f)$.

More concretely, the proof performs the following steps.

(1) For each n, form the absolute-error function $e_n = |f_n - f|$. From the full-set domination of f_n by g, derive a full-set bound of e_n by the nonnegative integrable function

$$H = |g| + |f|.$$

The appearance of $|f|$ is why the verified error majorant is not stated simply as $2g$ (Remark 4.3).

(2) From H, construct the dyadic smooth-level data needed by the profile argument. Internally, Theorem 5.1 returns a concrete exception sequence; Lemma 5.3 constructs a presented real apart from that sequence, and hence an admissible level at which the relevant upper and lower level sets are integrable by Theorem 5.2.

(3) The profile estimates at these levels yield Proposition 5.5: for a prescribed positive tolerance, there are an integrable set A, an index bound, and a positive measure threshold such that every sufficiently late error function has small integral outside each sufficiently large integrable subset of A.

(4) Convergence in measure (Definition 3.3) is invoked at the minimum of the profile threshold and a good-set bound. It supplies, for each sufficiently large n, an integrable good set $C \subseteq A$ on which f_n is close to f and whose complement in A has small measure. One then estimates $I(e_n)$ by its contribution on C plus its contribution outside C; this is the two-part estimate corresponding to Bishop and Cheng's Lemma 4.14.

(5) Finally, the constructive triangle inequality

$$|I(f_n) - I(f)| \leq I(|f_n - f|)$$

gives the conclusion of Theorem 4.1. □

Part II
The formalization and its audit

Three kinds of statement follow, and it is worth separating them before they are mixed. A *verified claim* is a named Lean declaration together with the axioms its proof term reports; the kernel decides it, and this paper only quotes the outcome. *Evidence* is what a script produced and what is shipped as a log or a checksum; it is reproducible, but it is evidence for exactly the property the script tests and for no more. A *limitation* is a claim nobody checked—the correspondence between a Lean statement and the mathematics it is meant to say, the manual review of the implementation, the transfer of a soundness theorem from Lean 3 to Lean 4. Section 10 states all three in that order, and the table at its head sets the second against the third row by row.

7 The Lean development

The development is carried out in Lean 4 [30], toolchain `v4.30.0`. The carrier of the presented reals is a concrete type,

`BishopCReal.CReal` = `RegularSeq`,

not a typeclass interface for an abstract constructive real-number structure; the rational layer is itself a quotient constructed inside the development. A quotient of the presented reals by Bishop equality, `BishopCReal.CRealQuot`, is also constructed, but it is used only as a transport device inside proofs: a ring identity between presented reals is discharged by `ring` in the quotient and the conclusion is pulled back to the representatives by `Quotient.exact`, so no statement on the

theorem surface is formulated about equivalence classes. Internally the quotient does occur in statements—the closure contains declarations about `CRealQuot` itself, in the layer that establishes its ring structure—but none of the public aliases, the reading layer, or the measure-theoretic development quantifies over equivalence classes. `Quot.sound` in the audit output comes from the kernel's quotient machinery: these two constructions, and function extensionality, which Lean derives from quotients and which the development uses through set extensionality. The data form of cotransitivity discussed in Section 2 is the declaration

$$\texttt{lt\_cotrans\_data} : \texttt{lt}\ a\ b \to \forall c,\ \texttt{PSum}\ (\texttt{lt}\ a\ c)\ (\texttt{lt}\ c\ b),$$

which returns the branch rather than only its Prop-valued disjunction. The displayed declaration is the field `BishopC.COF.lt_cotrans_data` of the ordered-field interface, instantiated at the rational layer; it is the only data-valued branch the avoidance construction consumes. Its presented-real counterpart, `regularSeqLtData_cotrans`, has the same shape one level up.

Integration spaces in the sense of Definition 2.1 are the structure `BishopSec1P.IntSpaceC X`, whose fields are

`L`, `I`, `L_resp`, `I_resp`, `add_mem`, `smul_mem`, `abs_mem`, `cutPos_mem`, `I_add`, `I_smul`, `cutNat_tendsto_raw`, `cutSmall_tendsto_raw`, `I_nonneg`, `continuity`,

corresponding to the axioms (A1)–(A6) as tabulated in Appendix A. The truncation field is `cutPos_mem`, which takes the positivity data `PosEventuallyData`; truncation at 0 is the derived lemma `cutZero_mem`, and the witness type `CutConstWitnessC` wires the evidence to the use sites (Remark 2.2). Integrable representatives, integrable complemented sets, relative integrals, profiles, and convergence in measure are represented by explicit structures or by Prop-facing predicates whose existential witnesses are opened only inside Prop goals. The final dominated-convergence statement is reached through an explicit majorant/profile route rather than by invoking a measurable-selection principle.

Theorem 4.1 is the implementation theorem

`BishopSec3P.bishop_cheng_thm_4_15_propC`,

exported under the paper-facing alias

`ChoiceFreeMeasureDCT.bishop_cheng_dominated_convergence_propC`.

The statement is short enough to display, and displaying it is the most direct answer to the question a reader of a formalization paper asks first—what, exactly, was proved:

```
theorem bishop_cheng_thm_4_15_propC
    {X : Type u} (S : IntSpaceC X)
    (fn : Nat -> IntegrableRepC3 S) (f : IntegrableRepC3 S)
    (hconv : ConvergeInMeasureC S (fun n => (fn n).toDataPFunRC) f.toDataPFunRC)
    (hdom : exists g : IntegrableRepC3 S, DominatedOnFullC fn g) :
    RepSeriesTendstoEpsPropC (fun n => (fn n).integral) f.integral
```

with the conclusion unfolding to the ordinary epsilon statement,

```
def RepSeriesTendstoEpsPropC (u : Nat -> CReal) (l : CReal) : Prop :=
  forall eps : CReal, regularSeqLtProp CReal.zero eps ->
    exists N : Nat, forall n : Nat, N <= n ->
      regularSeqLtProp (CReal.abs (CReal.sub (u n) l)) eps
```

Both are reproduced verbatim from the artifact. Note that f is an `IntegrableRepC3`: the limit is assumed integrable, as Remark 4.2 records. Its hypothesis (i) is `ConvergeInMeasureC`, which unfolds exactly to Definition 3.3; its hypothesis (ii) is `DominatedOnFullC`, which unfolds to Definition 3.4; and its conclusion is `RepSeriesTendstoEpsPropC`, the ordinary epsilon formulation displayed in Theorem 4.1.

7.1 The definitions the theorem is stated over

The theorem above is only as meaningful as the definitions it quantifies over, and the kernel does not check that those definitions say what this paper says they say. That correspondence is a limitation, recorded as such in Section 10; a reader who wants to audit it has to read the definitions. They are short, and they are given here so that the reading can be done without leaving the paper.

The integration-space interface has fourteen fields. Definition 2.1 states them as mathematics; here is the structure itself.

```
structure IntSpaceC (X : Type*) where
  L : Set (BFunC X)
  I : ∀ f : BFunC X, f ∈ L → CReal
  L_resp : ∀ {f g : BFunC X}, f ∈ L → BFunC.BEquiv f g → g ∈ L
  I_resp : ∀ {f g : BFunC X} (hf : f ∈ L) (hfg : BFunC.BEquiv f g),
    I f hf ≈ I g (L_resp hf hfg)
  add_mem : ∀ {f g : BFunC X}, f ∈ L → g ∈ L → BFunC.add f g ∈ L
  smul_mem : ∀ (a : CReal) {f : BFunC X}, f ∈ L → BFunC.smul a f ∈ L
  abs_mem : ∀ {f : BFunC X}, f ∈ L → BFunC.absf f ∈ L
  cutPos_mem : ∀ (a : CReal), PosEventuallyData a →
    ∀ {f : BFunC X}, f ∈ L → BFunC.minC f a ∈ L
  I_add : ∀ {f g : BFunC X} (hf : f ∈ L) (hg : g ∈ L),
    I (BFunC.add f g) (add_mem hf hg) ≈ CReal.add (I f hf) (I g hg)
  I_smul : ∀ (a : CReal) {f : BFunC X} (hf : f ∈ L),
    I (BFunC.smul a f) (smul_mem a hf) ≈ CReal.mul a (I f hf)
  cutNat_tendsto_raw : ∀ {f : BFunC X} (hf : f ∈ L)
    (hcut : ∀ n, BFunC.cutNat n f ∈ L),
    RepSeriesTendsto (fun n => I (BFunC.cutNat n f) (hcut n)) (I f hf)
  cutSmall_tendsto_raw : ∀ {f : BFunC X} (hf : f ∈ L)
    (hcut : ∀ n, BFunC.cutSmall n f ∈ L),
    RepSeriesTendsto (fun n => I (BFunC.cutSmall n f) (hcut n)) CReal.zero
  I_nonneg : ∀ {f : BFunC X} (hf : f ∈ L), BFunC.PointwiseNonneg f →
    RegularSeqLe zeroSeq (I f hf)
  continuity :
    ∀ {f : BFunC X} {fs : Nat → BFunC X},
      ∀ (hf : f ∈ L) (hfs : ∀ n, fs n ∈ L),
      (∀ n, BFunC.PointwiseNonneg (fs n)) →
      (hI : RepSeriesSum (fun n => I (fs n) (hfs n))) →
      CReal.ltE hI.sum (I f hf) →
      PointwiseSeriesBelowC fs f
```

Three things in it are worth pointing at. The integral `I` takes the membership proof as an argument, so it is defined *on* `L` and not on all of `BFunC X`; there is no junk value at a function outside the class. The truncation field `cutPos_mem` asks for $\min\{f, a\}$ only when a carries positivity data, which is what the source's clause (1) together with its own remark supplies, and no more (Remark 2.2). And `continuity` is the source's clause (2) — the constructive analogue of the continuity condition for a Daniell integral, which Definition 2.1 lists as (A6) — in the positive, point-producing form

the development uses: from a strict inequality between $\sum I(f_n)$ and $I(f)$ it *produces* a point at which the series stays below f. It is not a contrapositive; nothing is inferred from the failure of an existence claim.

The four notions the main theorem is stated over are equally short. A full set is one that contains the common domain of a sequence of integrable functions:

```
def IsFullC {X : Type*} (S : IntSpaceC X) (A : Set X) : Prop :=
  ∃ F : Nat → IntegrableRepC3 S, { x | ∀ n, x ∈ (F n).domain } ⊆ A
```

Domination is per-index, and this is where the hypothesis of this paper differs from the global one: the full set on which $|f_n| \le g$ holds is produced inside `RepAbsLeOnFullC` for each n separately, so it may depend on the index.

```
def RepAbsLeOnFullC
    {X : Type u} {S : IntSpaceC X}
    (u v : IntegrableRepC3 S) : Prop :=
  exists F : Set X, IsFullC S F /\
    forall x : X, x ∈ F ->
      forall (hudom : u.MemAt x) (hvdom : v.MemAt x)
        (hu : RepSeriesSum (fun k => u.valueAt x hudom k))
        (hv : RepSeriesSum (fun k => v.valueAt x hvdom k)),
        RegularSeqLe (CReal.abs hu.sum) hv.sum

def DominatedOnFullC
    {X : Type u} {S : IntSpaceC X}
    (fn : Nat -> IntegrableRepC3 S)
    (g : IntegrableRepC3 S) : Prop :=
  forall n : Nat, RepAbsLeOnFullC (fn n) g
```

Convergence in measure is stated against an arbitrary integrable set and an arbitrary positive ε, and returns for each large index a good subset whose complement in A has small measure and on which the functions are close:

```
def ConvergeInMeasureC {X : Type*} (S : IntSpaceC X)
    (fn : Nat → DataPFunRC X) (f : DataPFunRC X) : Prop :=
  ∀ (A : BishopC.BSet X) (hA : IntegrableSet1C S A)
    (eps : CReal) (_heps : regularSeqLtProp CReal.zero eps),
    ∃ N : Nat, ∀ n ≥ N,
      ∃ (B : BishopC.BSet X) (hB : IntegrableSet1C S B),
        (B.S1 ⊆ A.S1 ∩ f.domain ∩ (fn n).domain) ∧
        regularSeqLtProp ((IntegrableSet1_subC hA hB).rep.integral) eps ∧
        ∀ x (_hxB : x ∈ B.S1) (exf : f.domData x) (exfn : (fn n).domData x),
          regularSeqLtProp
            (CReal.abs (CReal.sub (f.toFun x exf) ((fn n).toFun x exfn))) eps
```

All five definitions are reproduced verbatim from the artifact, with doc-strings removed. Each unfolds to the corresponding definition of Part I: `IsFullC` to Definition 3.1; `RepAbsLeOnFullC` and `DominatedOnFullC` together to Definition 3.4, the first giving the per-index bound and the second quantifying it over the sequence; `ConvergeInMeasureC` to Definition 3.3; and `IntSpaceC` to the interface of Definition 2.1. Whether each unfolding is faithful is a judgement a reader makes by comparing the two, and it is the judgement the kernel cannot make.

7.2 Three-layer public interface

The public API has three layers:

Layer I: Type-valued convergence and profile data, data-carrying DCT route

Layer III: Prop-facing full-set DCT

⇓

Layer II: automatic dyadic smooth-level construction

The two columns are deliberately not joined by an arrow: Layer III is not obtained by applying the Layer I theorem, and Layer I is not obtained from Layer III either. The distinction between the layers is the distinction between a proof that merely asserts convergence and a proof that returns the modulus. In Lean, a statement in `Prop` is proof-irrelevant, and an arbitrary witness hidden in a `Prop`-valued existential cannot in general be eliminated into `Type` (data can nevertheless be recomputed when an independent decidable search produces a witness—Appendix B.12); a statement in `Type` is proof-relevant, and its inhabitants are data that later constructions may consume. A mathematician may read Layer III as the theorem and Layers I–II as the same theorem with the witnesses kept. More precisely, the Type-valued surface is a data refinement rather than an equivalent restatement: it demands and returns witnesses. A direct wrapper from the Prop-facing hypotheses to that surface would have to extract indexed witnesses hidden in `Prop` and would therefore require an additional choice principle. The artifact instead proves the Prop-facing and Type-valued routes separately; this is an interface and implementation claim, not a theorem of logical non-derivability between the two complete statements. "Returns data" means that a proof-relevant term exists; efficient program extraction is not claimed, and the fact that some public aliases are marked `noncomputable` is not evidence of choice either way (Section 10.1).

Layer I: Type-valued data-carrying route. The declarations

`ChoiceFreeMeasureDCT.l1_error_convergence_from_majorant_measure_convergenceC`

`ChoiceFreeMeasureDCT.integral_convergence_from_majorant_measure_convergenceC`

`ChoiceFreeMeasureDCT.dominated_convergence_from_error_majorant_profileC`

`ChoiceFreeMeasureDCT.dominated_convergence_from_pointwise_majorant_profileC`

`ChoiceFreeMeasureDCT.dominated_convergence_from_pointwise_majorant_good_set_profileC`

`ChoiceFreeMeasureDCT.l1_error_convergence_from_majorant_on_full_dataC`

`ChoiceFreeMeasureDCT.integral_convergence_from_majorant_on_full_dataC`

`ChoiceFreeMeasureDCT.dominated_convergence_from_pointwise_majorant_on_full_dataC`

consume explicit proof-relevant data: convergence moduli, good-set data, integrability witnesses, point-evaluation witnesses, profile/majorant data, smooth-level data, and Type-valued convergence output. In the full-set route, the carrier may depend on the sequence index. A Type-valued convergence interface is not claimed to be novel, since CoRN's `CvMeasure` is also Type-valued.

Layer II: automatic Type-valued wrappers. The public declarations ending in `_autoC` internally construct the dyadic smooth-level data by `BishopSec3P.lemma43DyadicSmoothDataC_construct`. They therefore avoid exposing an external `Lemma43DyadicSmoothDataC` argument. This includes `dominated_convergence_from_pointwise_majorant_on_full_data_autoC` and the variant combining explicit domination full sets with explicit convergence good sets; the two indexed set families remain distinct.

Layer III: Prop-facing theorem. The declaration `ChoiceFreeMeasureDCT.bishop_cheng_dominated_convergence_propC` accepts Prop-valued `ConvergeInMeasureC` and an existential integrable majorant satisfying `DominatedOnFullC`. It concludes Prop-valued epsilon convergence of the integral sequence. Existential witnesses from convergence and full-set domination are opened only inside Prop proof goals, not assembled into the selector required by the Type-valued full-set route. Consequently the final theorem does not manufacture a global Type-valued choice function. A separate Type-valued interface accepts precisely such data as `DominatedOnFullDataC`: for each index it contains a carrier, a proof that the carrier is full, and the domination proof on that carrier. It then returns `RepSeriesTendsto`, retaining the convergence modulus.

The two routes are therefore proved separately, and the separation is visible in the source. From the uniform-complement estimate onward the assembly exists twice, once producing a `Prop` and once producing a structure: some of the paired declarations differ only in interface plumbing—the type of the hypothesis, the type of the conclusion, how the witness is opened, how the result is assembled; others diverge further—the Prop-valued L^1 endpoint is an explicit proof, while its Type-valued counterpart composes data-valued lemmas. The two assemblies are structurally parallel rather than textually identical. What is not duplicated is the substance: the Lemma 4.3 apparatus—dyadic smooth-level data, level-set sequences, countable avoidance, the stratum where the known obstruction lives—exists once and is called by both. The repeated tail is an implementation consequence of the chosen interfaces: a direct wrapper would have to extract indexed witnesses hidden in `Prop`. The duplication makes that interface boundary visible, but is not by itself a proof that every choice-free implementation must duplicate the tail, or that the two complete statements are logically independent.

7.3 Formal counterparts of Section 5

Theorem 5.1 is exported as

`ChoiceFreeMeasureDCT.profile_smooth_away_from_sequenceC`

(implementation `BishopSec3P.thm_3_5_smooth_aeC`); the partition data underlying it, which is the source's Lemma 3.4 rather than its Theorem 3.5, is exported separately as

`ChoiceFreeMeasureDCT.profile_partition_dataC`

(implementation `BishopSec3P.lemma_3_4DataC`). Theorem 5.2 is exported in the source's global form as

`ChoiceFreeMeasureDCT.profile_level_sets_integrable_apart_globalC`,

implemented by `BishopSec3P.thm_3_6_all_posC`: it produces one exceptional sequence $T : \mathbb{N} \to \mathbb{R}$ good for every positive threshold at once, obtained as the source obtains it, by covering the positive half-line with the intervals $((n{+}2)^{-1}, n{+}2)$ and flattening the per-interval sequences. The interval-local core it is assembled from is exported as well, as `profile_level_sets_integrable_apartC`, since that is the form the dominated-convergence route consumes; it packages the conclusion that apart level thresholds give integrable upper and lower level sets with the expected profile measure. The word *apart* in those names is the load-bearing one: the thresholds are required to be apart from the explicit exception sequence returned by Theorem 5.1, and such thresholds are obtained by Lemma 5.3. Proposition 5.5 is exposed as

`ChoiceFreeMeasureDCT.uniform_complement_from_profile_levelsC`,

implemented by `BishopSec3P.lemma43UniformComplementData_of_majorantC`; the variant whose domination hypothesis is itself a full-set hypothesis, as in the statement of Proposition 5.5, is `lemma43UniformComplementData_of_majorantOnFullDataC`. Lemma 5.3 is formalized on the presented reals as the structure `Thm36B1ApartPointDataC`—a point t with $a < t < b$ together with, for every n, a witness that t is apart from the n-th term of the prescribed sequence—and its canonical inhabitant `thm36B1_apartPointDataC`, obtained from the data-valued nested-interval construction: the interval sequence, monotonicity of the endpoints, width shrinkage, Cauchy data for the left endpoints, and the limiting apart point. `thm36B_smoothPointDataC_construct` applies it to the profile exception sequence, and `lemma43DyadicSmoothDataC_construct` iterates that at the dyadic gauges to supply the smooth-level data consumed by Proposition 5.5; the latter is the declaration the public `_autoC` route calls. The generic-route counterparts `thm36B1_exists_apart_in_interval`, `thm36B1_apart_data` and `lemma_4_3_A_n_apart_data` are earlier declarations over an abstract ordered field and are not on the presented-real path taken by the public theorems. Step (4) of the proof in Section 6 is `lemma_4_14_local_two_epsilonC`, and step (5) is `integral_diff_lt_of_abs_error_integral_ltC`.

7.4 Artifact additions in this development line

This development line separates the public L1 error-convergence endpoint from the final integral-convergence endpoint. It adds the explicit full-set Type-valued route described above and a good-set convergence wrapper: the convergence data carries the good set, its integrability, membership in the ambient set, Type-valued point-evaluation witnesses, small complement measure, and the pointwise error estimate restricted to the good set.

The artifact further includes CReal-native support for the countable-avoidance construction used in the smooth-level interface. The formalized support constructs the data-valued nested interval step, the resulting interval sequence, left/right monotonicity, interval-width shrink, Cauchy data for the left endpoints, the limiting apart point, and the dyadic smooth-level data constructor.

`Mathdemo.ChoiceFreeDCTExamples` gives abstract application examples showing that both the explicit-smooth and automatic public aliases can be invoked as theorem inputs in proof terms. The file `Mathdemo.ChoiceFreeDCTConcreteExamples` adds a nonempty concrete zero-integral example on PUnit. It constructs the required good-set convergence and domination data and invokes the automatic good-set wrapper. This concrete example is intentionally degenerate—it is admissible precisely because Definition 2.1 imposes no normalization axiom (Remark 2.2). The file `Mathdemo.DiracIntegrationSpace` adds the normalized point-evaluation model, in which every clause of Definition 1.1 is discharged with no hypotheses, and which therefore also yields a model of Definition 2.1 through the adapter. It is a nondegenerate concrete model, although a simple one: the underlying functional is evaluation at a single point. `Mathdemo.FiniteWeightedIntegrationSpace` lifts that to finitely many points: for a list of points, repetition realizing integer weights, the functional $I(f) = \sum_{i \le N} f(p_i)$ discharges every clause of Definition 1.1 once a normalizing constant is given—a c whose $(N+1)$-fold sum is 1—and `twoPointDef11` instantiates it at two slots with the normalizing constant $1/2$. On a general carrier those two slots may coincide, so non-degeneracy is established at the closed instance on `Bool`: `boolIndicator_integral_pos` gives both Boolean singletons positive measure, and `boolTwoPoint_not_concentrated` shows the functional is the evaluation at neither point. The mathematical content beyond the Dirac model is clause (2): for a single point the continuity clause is the identity, while for several the point at which the series stays below the function has to be *found*. It is found by cotransitivity of the strict order along the finite sum, from the positivity witness carried by the strict inequality, with no appeal to choice. Finally, the module `Mathdemo.MathematicalInterface` restates the public surface in the vocabulary of Part I, as a

reading layer for a mathematician.

8 Relation to the no-go results

The constructive content of this section is written out, from the side of the constructions, in Appendix B.5. The core of the answer is a conjunction of two conditions: (a) the input to the avoidance step is at hand as data—the explicit family that the source's own proof constructs—rather than a "countable set" hidden behind a proposition; and (b) the avoidance construction for that sequence (nested intervals) can be carried out over the order data of presented reals. The explicit-sequence form by itself does not evade the obstruction: over an abstract real-number object, a general principle taking an explicit sequence and returning a real apart from all of its terms would itself be a sequence-avoidance principle of exactly the kind the no-go results concern. What makes the avoidance possible is the restriction of the object, (b); condition (a) is the formal precondition that makes it applicable.

The route formalized here passes through the profile theorem, and the profile theorem is precisely where the choice-free programme is known to be obstructed. A choice-free claim for this route must therefore specify the real-number object and mathematical interface relative to which it is made.

Spitters reports Richman's formulation that measure theory and the spectral theorem are the major challenges for a development without countable choice [22, §1], attributing it to a 2005 note of Richman's that we have not consulted; Richman's published case studies of that programme are the Hilbert syzygy theorem, trace-class operators, and the fundamental theorem of algebra [20], the last of which he carried out in full without choice [18]; the weak choice principles that remain in play are analysed in [19]. Spitters took up Richman's challenge for integration theory [22]. In the choice-free version he observes that the profile theorem—the positive form of the classical fact that an increasing function on the reals has at most countably many discontinuities—*implies the uncountability of the reals*: for the integration space

$$I(f) \;=\; \sum_n 2^{-n} f(q_n)$$

determined by a sequence (q_n) of points of $[0,1]$, and for $f = \mathrm{id}$, a smooth point of the profile of f must be distinct from—in his sense, which is positive distance, hence apart from—every q_n. He then exhibits a sheaf model over $\mathbb{R}$, built from a sequence of linear functions forming a dense 45° lattice—that is, lines of slope ± 1—in which the statement "for every sequence of reals there is a real apart from all of them" fails, and concludes that a proof of the profile theorem without choice is not to be expected; his development consequently obtains the approximation results it needs from a pointfree Stone representation theorem instead of from Bishop and Cheng's approximation theorem. What the note claims for itself is worth stating precisely, since it bounds what is already settled: Spitters writes that there is no hope of proving the profile theorem without choice, and of its approximation consequence that it is not clear to him whether it can be proved without choice; what the pointfree route supplies is that consequence, not the theorem [22]. Petrakis and Zeuner record the same obstruction, and their formulation of what remains open is broader than a single theorem: they ask whether the basic *applications* of profile theory can be recovered inside predicative Bishop–Cheng measure theory through choice-free variants of its basic notions and results [24, §11]. One level down, Lubarsky shows that intuitionistic set theory without choice does not prove that the constructive Cauchy reals are Cauchy complete, and records Richman's observation that what is actually provable is the completeness of *sequence–modulus pairs* rather than of the reals themselves [21]. In his later survey the alternative is named in passing: excluded middle

enters that construction as a case split "because the alternative is to use Countable Choice, and most would automatically avoid that as just too dishonest", and formulations of "almost Cauchy" that are "mutually equivalent under Countable Choice" are shown there to come apart without it [13, §22.2.2, §22.7]. Distinctions of that kind are invisible while choice is available; the profile route is one more place where withdrawing it makes one visible. Richman puts the resulting order of motivations in its strongest form: "I have come around to thinking that sequences are undesirable, therefore I don't want to use countable choice, rather than that I don't want to use countable choice, therefore I must abandon sequences" [11, §19.2]. Behind all of it stands Diaconescu's theorem: in a topos, where AC reads "Every epimorphism has a section", "an intuitionistic model of set theory with the axiom of choice has to be a classical one" [6]. Choice in that form is therefore not a convenience a constructive development may help itself to, and a foundation may be designed to *lack* it rather than merely to avoid using it.

The present formalization neither refutes these results nor settles the open problem. It is compatible with them, and the mechanism is worth stating precisely, because it is the content of the word "presented" in the title.

(i) **The reals are presented.** A real is a regular sequence of rationals, so it carries its own modulus. Given $a < b$ and c, the cotransitivity split needed by the construction is returned as data from rational approximations. Bishop's nested-interval argument (Lemma 5.3) is therefore available as a *construction*, not as a choice principle. The artifact formalizes it and audits it with the same axiom output as the rest of the development. The mechanism by which the branch is returned as data—the one-point certificate and strong gauge extraction—is written out in Appendix B.12. Maietti and Sambin diagnose the same step from the other side: turning a formal point of the formal topology of the reals into a Cauchy sequence needs dependent choice in the setoid model over Martin-Löf type theory, and in a foundation without choice "this proof cannot be carried out"—because "what actually happens in MLTT is that the splitting of points is already given with an operation choosing an interval where the point is, and hence from this choice a definition by cases can be given similarly to that done classically" [12, §20.4.3]. Here that operation is not inherited from the ambient type theory but supplied, as part of the presentation.

(ii) **The uncountability step is discharged, not avoided.** Theorem 5.1 as formalized does not assert that the set of non-smooth levels "is countable" with an existentially hidden enumeration; it returns an *explicit exception sequence* and concludes smoothness at every level apart from it. The countable-avoidance construction of (i) is then applied to that exception sequence to obtain an admissible level threshold. In short, the development pays exactly the price that Spitters identifies, and pays it constructively over presented reals.

(iii) **Completeness is used only in representation-carrying form.** Limits are constructed from an explicitly supplied modulus and are never extracted from a bare Cauchy property. This is precisely the distinction that Lubarsky attributes to Richman, and it is why his independence result does not bear on the development.

(iv) **Scope of the claim.** Both Spitters and Lubarsky note that, in the relevant models, the Cauchy reals and the Dedekind reals need not coincide [22, §2]; in the models Lubarsky cites, even the smallest Cauchy-complete set of reals can be a proper subset of the Dedekind reals [21, §7]. We do not undertake here a model-theoretic analysis of which real-number object the counterexample family inhabits. Our claim is the correspondingly modest one: *over presented reals, and for the explicitly quantified integration-space interface of Definition 2.1,*

the Bishop–Cheng profile route can be carried out in Lean's type theory with no appeal to Lean's `Classical.choice`. This is not a claim that the profile theorem is provable without choice over an arbitrary constructive real-number object; it is not a predicative result; and it does not address the open problem of [24], which concerns the recovery of the applications of profile theory inside a predicative setting.

Two limits on the phrase "choice-free" should be recorded here. A foundation may be built to *lack* choice: the Minimalist Foundation is "characterized by the lack of whatsoever choice principle, including the so-called axiom of unique choice" [12, §20.1], and in it the reals form no set. Lean is not such a foundation—unique choice is available once `Classical.choice` is admitted—so the audit certifies not that no choice principle is valid but that the named declarations do not *invoke* one. That weaker property is the checkable one, and it is checkable because Lean's `Exists` is `Prop`-valued rather than a Σ-type: the propositions-as-sets identification that makes choice provable in MLTT is unavailable, so extraction requires the axiom and its absence is mechanically detectable (Section 10.1). Second, realizability is not elimination. Berger and Seisenberger prove countable and dependent choice realizable and then withhold the conclusion, "since in their proofs the very same principles are used", adding that one cannot expect elimination for "choices of elements in abstract domains for which no computational methods are available" [17, §29.3]. That last clause names the hypothesis replaced here: over presented reals the domain is not abstract, and the branch is computed from rational approximations.

9 Related work

Bishop–Cheng measure theory and its reconstructions

Bishop and Cheng's memoir *Constructive Measure Theory* is the mathematical source for the measure-theoretic route formalized here [2]; the surrounding constructive analysis is that of [1, 3], and the integral-first tradition it belongs to goes back to Daniell [4]. Chan's recent monograph develops constructive probability theory on the same basis and contains an extended treatment of integration and measure [5]. Coquand and Palmgren give a pointfree, measure-algebraic account [23]. Ciraulo develops a more recent pointfree route toward measure theory, including a measure on σ-sublocales; that chapter uses countable choice freely, and notes that carrying the treatment over to choice-free frameworks would require a different definition of the frame of reals [10].

Petrakis and Zeuner have taken, in their own words, the first steps towards a predicative reconstruction of Bishop–Cheng measure theory: they replace the quantification over proper classes in the L^1 construction by set-indexed families of partial functions, and prove that the resulting family of canonically integrable functions is itself a pre-integration space and an appropriate completion of the original one—so the completion is recast predicatively rather than discarded [24]; their development also avoids countable choice. The present work does not claim a predicative reconstruction. The ambient theory has an impredicative universe of propositions: a $\forall$ or an $\exists$ whose body is a proposition is itself a proposition, whatever is quantified over, so that no universe is climbed. Sets are accordingly $X \to$ `Prop` and separation is available impredicatively, and no attempt has been made to check that each definition of the development stays inside a predicative fragment. Fullness, the order relations on presented reals, and the Prop-facing chain of Section 7 are propositions; several of these notions also have Type-valued forms, which live in the predicative universes—the data form of the order, and integrable sets, which are structures. The presence of those forms does

not make the development predicative, since the impredicative universe is available throughout.

That disclaimer is compatible with a more precise description of the relationship: the two contributions are complementary, not merely disjoint. First, the Type-valued face of the principal theorem chain carries as data, in the predicative Type universes, exactly what has to be produced: convergence moduli, the carrier chosen at each index, and integrability representations. The remaining fields of those records are propositions—that the carrier is full, that the domination inequality holds on it, and that the terms are close at each gauge—so the Type-valued route is not free of `Prop`; what it does not contain is a step that extracts a witness from a `Prop`-valued existential by a choice principle. Fullness is one of those propositions, and the sharpest case: `IsFullC` is a `Prop`-valued existential over sequences of integrable representations, and the data-facing structures carry a proof of it rather than exposing its generating sequence. That is consistent with the preceding sentence rather than an exception to it: the existential is destructured only while proving `Prop`-valued facts about full sets—closure under intersection, and the passage to a single integrable representative—and is never eliminated into a `Type`-valued result or selector. The chain's only passages from propositions to data are explicit supply of data and decidable search. The negative half of this is certified mechanically by the choice-free audit: in Lean, unconditional extraction of data from a `Prop`-valued existential requires `Classical.choice` (Section 10.1), so an axiom output confined to `[propext, Quot.sound]` proves the absence of choice-based extraction throughout the printed declarations and everything their proof terms depend on; Section 10.1 delimits what covers the rest. The positive classification—that every passage from propositions to data proceeds by explicit supply of data or by decidable search—comes from source inspection, and its principal instance is written out in Appendix B.12. Second, integrable functions are handled as representation data from the outset and never pass through the impredicative L^1-completion; this runs in the same direction as Petrakis and Zeuner's central device of recasting the completion through set-indexed families—and it was they who carried that device out, as theorems, inside a predicative foundation.

The two contributions therefore divide the two known obstacles of the memoir between them: predicativity is addressed by Petrakis and Zeuner at the level of the theory—in their own words, the first steps towards a predicative reconstruction—while freedom from choice is established here, at the level of machine-checked proofs, for a Lean development over presented reals (for the machine-checked precedent in Coq, see the CoRN comparison below). Determining, inside Lean, which fragments of the present development admit a predicative reading is left as future work, alongside their open problem (Section 8).

Integration in proof assistants

Sacerdoti Coen and Tassi formalized a constructive proof of Lebesgue's dominated convergence theorem in Matita [28]. Boldo, Clément and Leclerc formalized the Bochner integral in Coq [29]. The repository whose measure-theoretic files are audited below is itself built on an earlier formalization of Bishop-style real analysis in Coq, reported by Cruz-Filipe, Geuvers and Wiedijk as C-CoRN [38]; we cite it as the provenance of the repository and have not consulted it for the comparison made here, which is against the pinned snapshot itself.

One of the most substantial formalizations of Bishop-style analysis in a proof assistant is Bickford's: "We have formalized in Nuprl all of Chapter 2 of Bishop and Bridges, 'Constructive analysis'" [16, §28.1]. That report is a particularly relevant neighbouring formalization on the mathematical side, and it also marks that boundary. Its concluding section states which part of the programme was carried out and which was not: the numerical-analysis methods "can be adapted to yield provably correct, efficient algorithms for operations on Bishop's real numbers. We have illustrated how this can be done for the functions defined in Chapter 2 of Bishop and Bridges, with

the notable exception of general definite integrals" [16, §28.14]. Bickford's target is thus Bishop–Bridges analysis rather than the Bishop–Cheng memoir, and the general integral is explicitly outside what was achieved there; the route formalized in the present paper begins on the other side of that line. One further observation of his bears on a choice made here: a first attempt that generalized Bishop's regular sequences to admit arbitrary moduli was withdrawn—"we found that this choice of representation was a mistake" [16, §28.2]—in favour of the source's own fixed convention, which is the convention kept here (Section 2.1). He also separates constructive existence from executability, reporting a modulus obtainable in principle from Brouwer's principles that the extracted code could not compute in practice [16, §28.13]; the audit of Section 10.1 concerns axiom dependence, not running time, and no performance claim is made here. On the classical side, Lean's `mathlib` contains a dominated convergence theorem for the Bochner integral over a measure space [35, 36]; that development is classical and is not comparable to the present one at the level of hypotheses. The closest prior formalization of the Bishop–Cheng route itself is Séméria's, discussed below.

Constructive reals in proof assistants

Miyamoto's survey of exact real arithmetic is the closest published account of the representation used here—its Definition 27.1 is the same object, with the regular sequences as the case in which the modulus is the identity—and its treatment of uniformly continuous functions makes the design choice of the present artifact explicit: rather than invoking the fan theorem to obtain the missing bounds, "we require additional information to compute the lower and upper bounds ... so that we don't rely on the fan theorem" [15, §27.2.1, Def. 27.13]. Replacing an appeal to an extra principle by an extra field of data is the move the presented reals make for cotransitivity. In the theory of computable functionals behind Minlog the analogue of the `Prop`/`Type` split appears as an explicit declaration: predicates "can be declared to be either computationally relevant (c.r.) or else noncomputational (n.c.)" [14, §26.3], and the extracted term is determined by that declaration.

Geuvers and Niqui axiomatized the constructive reals of the FTA project—the development that later grew into CoRN—and proved the axiomatization categorical [26]. Séméria's JFLA paper discusses Cauchy-data constructive reals, setoid equality, and the distinction between the constructive and classical real-number layers in Coq's constructive-real infrastructure [27]; it is cited here for that infrastructure, not as a source for the dominated-convergence theorem.

Booij's locators [25] approach the same proposition/data axis from the opposite side of the representation divide. There the reals are the extensional Dedekind reals of univalent type theory, carrying no presentation, and computational content is carried per real as additional structure: a *locator*, which replaces the disjunction in the locatedness clause $\forall q < r.\, (q < x) \vee (x < r)$ by an untruncated sum. His Theorem 3.9.5—for a real number, locators and signed-digit representations are interdefinable—is the dictionary between the two sides at the level of structure; read through it, the presented reals used here live on the locator-carrying side. (His Theorem 3.9.7 states the corresponding equivalence for *mere* existence: that a locator exists, that a signed-digit representation exists, that a rational Cauchy sequence with limit x exists, and that x is a Cauchy real, are equivalent. He reserves "to have" for untruncated structure and "to exist" for the truncation, and item 1 is written $\|\mathrm{locator}(x)\|$.) The exact correspondence—the same decidable-search engine, the same normal form as the one-point certificate, dual answers to the same extensionality obstruction, and the same strategy of replacing choice principles by carried structure—is set out in Appendix B.13. Among proof-assistant neighbours his theory thus occupies a third coordinate, the extensional side with carried structure, distinct both from CoRN's setoid representations and from the raw presentations used here.

The closest prior formalization: CoRN

The closest prior proof-assistant formalization identified for Bishop–Cheng dominated convergence is Séméria's Coq/Rocq development in CoRN [37]. The audited community-repository snapshot is CoRN commit `ada7c0b497ff15dd67cf7932c6f20e143a2aee2f`. At that commit, the pinned source file `CMTMeasurableFunctions.v` contains `CvMeasure`, the source's Lemma 4.14-style declaration `DominatedMeasureCvZero`, and `DominatedConvergence`. The reproduced Rocq 9.0.1 audit packaged with this artifact reports all three declarations as `Closed under the global context`. For each declaration, `Print Assumptions` listed no global axioms in its dependency closure. This describes the declarations' global axiom footprint; it does not make the theorems free of mathematical hypotheses. Their statements remain parameterized by the explicitly quantified `IntegrationSpace` and `ConstructiveReals` structures, whose fields specify the mathematical setting in which the results are proved. The same reading applies symmetrically to the Lean audit reported here, whose statements are parameterized by the integration space of Definition 2.1.

CoRN also makes the countable-avoidance step explicit and data-valued. In the pinned `CMTprofile.v`, `CRuncountable` and its diagonal variants return a real in a prescribed interval apart from every member of a supplied sequence. The three-index variant feeds the jump-point enumeration used to prove integrability of level sets away from an explicit exception sequence. Its proof covers the supplied sequence by open intervals of controlled total measure in a concrete real-line integration space and extracts a point from the positive-measure remainder. The present Lean development instead implements Bishop's nested-interval construction (Lemma 5.3) over fixed regular-sequence presentations. Thus both developments discharge the same countable-avoidance obligation by explicit constructions, but by different routes. The two routes also differ in the depth of their prerequisites: the covering argument runs inside a concrete real-line integration space, at a stage where the measure apparatus is already constructed, whereas the nested-interval construction uses only the order structure and rational approximations of the presentations, before any measure theory is in place. The same obligation is discharged at different depths of the respective developments; the contrast is a difference of position, not a ranking.

9.1 Comparison of domination assumptions

The theorem interfaces also differ materially. Both final DCT routes assume convergence in measure, not pointwise convergence. CoRN's `CvMeasure` hypothesis is Type-valued, and `partialFuncLe` states domination on the intersection of the partial-function domains. The Lean artifact exposes both a Type-valued full-set route, with an explicit indexed selector of full carriers, and a Prop-facing full-set theorem. CoRN quantifies over an abstract `ConstructiveReals` implementation, whereas Lean fixes regular-sequence presented reals; CoRN's `IntegrationSpace` also contains a distinguished integrable unit of integral one, whereas Definition 2.1 has no normalization field and admits the degenerate model described in Remark 2.2. These differences are not ordered by logical strength, and neither complete theorem surface is claimed to be a formal generalization of the other.

Concretely, CoRN's public `DominatedConvergence` assumes global pointwise domination:

```
forall n, partialFuncLe (Xabs (fn n)) g.
```

The Lean artifact exposes the full-set hypothesis in two forms. The Prop-facing theorem assumes that for each n, domination holds on a full set, which may depend on the index (Definition 3.4). The Type-valued theorem takes those carriers, fullness proofs, and bounds as the explicit selector `DominatedOnFullDataC` and preserves a convergence modulus in its conclusion. The formal bridge

`DominatedOnFullDataC.ofGlobal` packages every global pointwise domination family as full-set data; no converse is asserted. Thus, relative to the domination clause of CoRN's public theorem surface alone, full-set domination is a weaker hypothesis, while the Type-valued presentation asks the caller to supply more reusable proof-relevant data than the Prop presentation. (By Bishop and Cheng's own notational convention the domination hypothesis of their Theorem 4.15 is also a full-set condition—Remark 4.3; the comparison in this paragraph is with the CoRN declaration, not with the source.)

We do not claim a formal strict generalization of the CoRN theorem, because the ambient real-number and integration-space interfaces differ. CoRN's `CR_cv` and the Lean Type-valued full-set conclusion are both data-valued, but the two complete theorem statements are not ordered solely by logical strength. Across the Lean pointwise-majorant wrappers, the verified error majorant is $g + |f|$ or $|g| + |f|$, depending on the route, not $2g$, unless a separate proof of $|f| \leq g$ is supplied.

A summary comparison:

Aspect	This development	CoRN
Reals	presented regular sequences	abstract `ConstructiveReals`
Integration space	Definition 2.1, no normalization	abstract `IntegrationSpace`, unit of integral 1
Convergence API	Type layers + Prop surface	Type-valued `CvMeasure`
Domination	per-index full set	global pointwise
Countable avoidance	nested intervals over presentations	interval cover of controlled measure
Audit output	`[propext, Quot.sound]`	`Closed under the global context`
Comparison caveat	not strictly ordered	not strictly ordered

The apparent asymmetry of the audit outputs—`[propext, Quot.sound]` on the Lean side against `Closed under the global context` on the Rocq side—should not be read as a comparison of axiom counts. As Carneiro's axiomatization of Lean records [33], Lean builds quotient types into the kernel, with definitional computation rules and the axiom `Quot.sound`, and derives function extensionality from quotients together with propositional extensionality; Coq does not build quotient types into the kernel with definitional computation rules, and its ecosystem compensates with setoids; Carneiro notes that quotients of sets can be constructed in Coq, but without computation rules. At least one of the two Lean axioms thus pays, as an axiom, for what the Coq side obtains from setoid machinery and quantified interfaces. The shape of the audit output reflects a difference in foundational design, not a difference in mathematical strength.

Accordingly, we are not aware of another choice-free formalization of the Bishop–Cheng dominated convergence theorem in Lean 4; mathlib's dominated convergence theorem is classical. This records what we looked for and did not find, and is not a claim of priority of any kind—not to the first proof-assistant formalization, the first constructive one, or the first choice-free one; the closest prior work we did find is set out above. What was searched is stated rather than left implicit: the two published surveys below, the CoRN repository at the pinned commit, the constructive real-number modules of the Coq standard library that CoRN's interface is instantiated by [31], and `mathlib`. That is a bounded search, not an exhaustive one, and the surveys are consistent with the statement without establishing it: Miyamoto's list of Bishop-style formalizations records Nuprl, CoRN and Minlog and does not include Bishop–Cheng measure theory [15, §27.3.4], and Bickford, reporting one of the most substantial of those developments, states that the general definite integral was the one part of his programme not carried through [16, §28.14]. The contribution is a Lean 4 realization over presented reals, Type-valued global and full-set theorem routes, automatic smooth-data wrappers, a Prop-facing full-set theorem, a Lean implementation of countable avoidance via Bishop's nested-interval construction, and a reproducible Lean audit whose named public declarations have declaration-level axiom output confined to `[propext, Quot.sound]`.

9.2 What is different here, and what is not

The comparisons above are scattered through the section, and a reader is entitled to ask what they add up to. They add up to three differences and two non-differences, and it is worth being explicit about which is which, because the theorem being proved is the same theorem.

The hypothesis is weaker on domination. CoRN's public `DominatedConvergence` assumes `forall n, partialFuncLe (Xabs (fn n)) g`: one bound holding wherever the partial functions are defined. The hypothesis here is that for each n the bound holds on *some* full set, and that the set may depend on n (Definition 3.4, and `RepAbsLeOnFullC` in Section 7.1). The difference is narrower than it looks, and it is worth narrowing it here rather than leaving a referee to do it. CoRN's integrable functions are dependent pairs carrying a `PartialRestriction`, so each `fn n` already comes with its own domain, and `partialFuncLe` quantifies over the points where both functions are defined; the domain that varies with the index is therefore present on the CoRN side too. What differs is the form in which it enters: there it is the domain the representation supplies, here it is a full set produced existentially by the hypothesis and consumed as data. We do not claim that this orders the two theorem surfaces by strength, and the rest of this section gives the reasons they are not ordered.

The avoidance is discharged before measure theory, not inside it. Both developments make the countable-avoidance step explicit and data-valued, and neither appeals to choice for it. But CoRN's `CRuncountable` covers the supplied sequence by intervals of controlled total measure in a concrete real-line integration space and extracts a point from the remainder, so the step runs at a stage where the measure apparatus is already built. The construction here (Lemma 5.3, Appendix B.3) uses only the order structure of the presentations and their rational approximations, and closes before any measure theory is in place. The consequence is not that one is better: it is that the obligation which the no-go results attach to this route is discharged here at the level of the reals themselves, where the presentations are, rather than at the level of a measure space built on top of them. That is why the presented-real hypothesis does the work it does.

Propositions and data are separated by design rather than by convention. This is not what distinguishes the present development, and it is worth saying so plainly. CoRN states `CvMeasure` Type-valued, and the `ConstructiveReals` interface it quantifies over already draws the distinction at the level of the reals: its strict order `CRlt` is `Set`-valued, its linearity field returns a sum rather than a disjunction, and `CRltEpsilon` and `CRltForget` are fields naming the two directions of the passage between the propositional and the data form of that order [31]. What is different here is where the separation is carried: the development proves two routes with the same informal dominated-convergence assertion and keeps them apart at the level of the theorem rather than only of the reals: a Type-valued route in which the carriers, the fullness proofs and the bounds are an explicit selector, and a Prop-facing theorem whose hypotheses are propositions and whose conclusion is the ordinary $\forall\varepsilon\,\exists N$ statement. Section 7.2 sets out the three layers this produces. The point of the separation is that every crossing from a proposition to data becomes a place one can look at, and Appendix B.12 writes out the principal one; a development that does not separate them has the same crossings but no place to look.

The reference is internalized. CoRN quantifies over an abstract `ConstructiveReals` interface whose implementations live in the Coq standard library. A theorem stated over that interface does not depend on any particular implementation—that generality is what the abstraction buys, and it is a generality the development here does not have—but the abstract real-number theory it is proved against does sit outside the artifact, and `Print Assumptions` descends into it. Here the presented reals are a concrete project-local type inside the development, which is why the size reported in Section 10.2 is what it is: that layer is inside the audited project closure rather than

borrowed from a library. It is not a claim of self-containment: Lean and `mathlib` remain external trusted dependencies, as Section 10.1 records.

The theorem is not new, and neither realization generalizes the other. The mathematics is Bishop and Cheng's, and their proof is the proof followed here. The two theorem surfaces live over different real-number objects and different integration-space interfaces – CoRN's carries a distinguished integrable unit of integral one, Definition 2.1 carries no normalization field – and neither is a formal generalization of the other. No priority is claimed, in either direction. What is claimed is the three differences above, each of which is a difference in what the formalization commits to rather than in what the theorem says.

What the paper contributes, stated affirmatively. The qualifications above are numerous, and it is fair to collect what survives them in one place. (1) A Lean 4 artifact of 191 files in which the Bishop–Cheng dominated convergence theorem is proved over a concrete presented-real type, with 21 public aliases whose kernel axiom footprint is `[propext, Quot.sound]`. (2) A choice-free countable-avoidance construction with the cut points fixed in advance (Appendix B.3), discharging in the order theory of the reals the step the no-go results obstruct for unrestricted constructive reals. (3) A hypothesis-free adapter deriving every field of Definition 2.1 from a clause-by-clause transcription of the source's Definition 1.1 (Remark 2.2), so that the interface the theorem is stated over is tied to the memoir rather than asserted to match it. (4) An audit apparatus that reports what it does not cover: a vacuous-statement check, a reachability report, a frozen-name check, and a reproduction of the closest prior work's assumptions audit in the other proof assistant, all scripted and shipped. (5) A record, in Appendix B.11, of which parts of the formalization were transcription and which were work the source leaves implicit. None of these is a new theorem. All of them are checkable, and the checks are in the deposit.

10 Trust boundary and reproducibility

The four parts of this section answer four separate questions, and they are kept apart because the answers are of different kinds: what is checked (§10.1), what the checks are checks *of* (§10.2), what is not claimed (§10.5), and how a reader reruns the evidence (§10.6).

The table below is the single place where the figures of this section are stated together with what produces them and, in the last column, what they leave open. The last column is the point of the table: each row is evidence for something narrower than the sentence it supports, and the difference is where a reader should look for what is still taken on trust.

What is claimed	Evidence	Regenerated by	What it does *not* establish
The named public declarations depend on `[propext, Quot.sound]` only	`logs/build_audit.txt`	`./build_audit.sh`	Nothing about declarations it does not name, and nothing about the *statements* being the intended ones—that is the source comparison of Appendix A.
The 104 reading-layer declarations split 102/2	`logs/reading_layer_axioms.txt`	`./build_audit.sh`	That the layer is complete: it is the layer this paper defines, not a canonical one.
No forbidden token occurs in the source closure of the six audit roots (191 files)	`logs/static_audit.txt`	`tools/static_no_choice_audit.py`	Anything about files outside that closure, and anything a token scan cannot see—a proof term is not read by it.
2,552 of 5,431 declarations are reachable from the claim-carrying modules	`logs/reachable_core.txt`	`tools/reachable_core.lean`	That the remaining 2,879 are individually necessary; see §10.2.
No declaration in the source has a conclusion reducing to `True`, beyond eight recorded ones	`logs/vacuous_statements.txt`	`tools/vacuous_statement_check.lean`	That a statement is the intended one: a non-vacuous conclusion can still be the wrong theorem. See §10.5.
The shipped bytes are the audited bytes	`SHA256SUMS`	`sha256sum -c`	That the tree was built from them on the reader's machine; that is what rerunning the audit shows.
CoRN reports `Closed under the global context`	`audits/corn/`	`audits/corn/commands.sh`	A comparison of axiom strength between the two systems; see §9.

10.1 Choice-free audit

The development declares `mathlib` [35] as a dependency, and `mathlib` modules do occur in the transitive import closure of the public roots. Importing is not the same as depending on a result: Lean's `#print axioms` follows the dependencies actually used by a declaration rather than every axiom made available by its imports. The real-number and measure-theoretic layers used above are constructed inside the artifact; in particular, the audited declarations do not consume `mathlib`'s `Real` or `MeasureTheory` hierarchy. This source-level separation, together with the declaration-level audit, explains why the reported axiom output is compatible with the presence of `mathlib` in the import closure.

The claim is restricted to the public theorem aliases, the named implementation declarations, and the public transitive import closure of `choicefree-bc-dct`; it is not a claim about any file outside that closure. The build script prints axioms for the public declarations and runs a static audit after stripping Lean comments. `Quot.sound` is expected from the quotient constructions described in Section 7—the presented rational layer and the transport quotient of the presented reals; quotient extraction and classical selector axioms are not expected in the audited declarations.

What the claim is relative to. Collecting the qualifications made above, the audited statement is: for every type X and every integration space S on X in the sense of Definition 2.1, the named public declarations hold, and their proof terms use no axioms beyond `propext` and `Quot.sound`. The quantified structure S plays the same role that `IntegrationSpace` plays in CoRN, and the reals are the concrete presented reals of Section 2 rather than an abstract real-number interface. The countable-avoidance step required by the profile route is discharged inside this audit, not assumed

(Section 8). What is *not* claimed is that Definition 2.1 is itself a rendering of Bishop and Cheng's Definition 1.1 clause for clause (a clause-by-clause transcription is supplied separately, together with a hypothesis-free adapter deriving every field of Definition 2.1 from it—Remark 2.2), nor a model over a continuum, nor one realizing arbitrary positive weights, nor a predicative development, nor any statement about real-number objects other than the presented reals.

The CoRN assumptions audit and the Lean axiom audit are different mechanisms in different proof assistants. The CoRN audit reports `Closed under the global context` for the inspected declarations. The Lean audit reports `[propext, Quot.sound]` for the named public declarations. Both statements are relative to the explicit theorem interfaces being audited.

Three complementary checks

The audit is deliberately not based on a single text search.

(1) **Kernel dependency output.** Dedicated Lean files invoke `#print axioms` on the public aliases and the implementation declarations used by them. The reported axiom names are confined to the set {`propext`, `Quot.sound`}, and the principal public DCT declarations report both. In particular, the fact that an alias is marked `noncomputable` is not treated as evidence for or against choice; the criterion is the dependency output of its elaborated term.

(2) **Transitive source-closure scan.** `tools/static_no_choice_audit.py` computes the import closure of six public roots, strips Lean comments, and rejects forbidden tokens in the resulting source: the selectors (`Classical.choice`, `Classical.choose`, any `Classical.`, `native_decide`, `Quot.out`, `Quotient.out`, `open Classical` and the `classical` tactic), the placeholders (`sorry`, `admit`), the escapes from kernel checking (`unsafe`, a declaration-initial `axiom`, `partial def`, `implemented_by`, `extern`, `debug.skipKernelTC`), and a conclusion of `True`. Seven were added in v0.4.2—the declaration-initial `axiom`, `Quotient.out`, `partial def`, `implemented_by`, `extern`, `debug.skipKernelTC`, and the `True` conclusion; `unsafe` was scanned from the outset. Each reports zero on the closure. The recorded run traverses 191 Lean files and passes (`closure_files: 191`, `tracked_lean_files: 191`, `STATIC AUDIT PASSED`). The audited closure and the Lean paths recorded in `SHA256SUMS` are compared as sets in both directions—excluding the Lean files under `tools/`, which are audit instruments rather than part of the development and are therefore outside the closure—and the run fails if they differ, or if an import under a project-local prefix is absent from the tree, or if an import is neither project-local nor on the allowed-external list: a missing file is a packaging error and must not be read as an external dependency, which would silently shrink the closure being audited. Because the audit roots include the aggregation module `Mathdemo`, this closure covers the four reading-layer modules: the mathematical reading interface `Mathdemo.MathematicalInterface`, the Definition 1.1 transcription `Mathdemo.SourceIntegrationSpaceDef11`, and the two concrete models `Mathdemo.DiracIntegrationSpace` and `Mathdemo.FiniteWeightedIntegrationSpace`. The scan complements the declaration-level check by detecting suspicious source in the public closure even when it is not reached by one selected theorem.

(3) **Integrity and cross-system provenance.** Root checksums cover the tracked artifact, while a separate checksum file protects the reproduced CoRN audit package. The CoRN commands pin both the source commit and Rocq version. Thus the Lean claim and the prior-work comparison can be reproduced independently rather than inferred from prose in this paper.

The checks answer different questions. A source scan cannot replace kernel dependency analysis, because the presence of an imported classical declaration does not show that a proof term uses it. Conversely, a short list of printed axioms does not by itself establish that the intended source tree, versions, and audit inputs were the ones inspected. The artifact therefore reports both kinds of evidence and protects their inputs with checksums.

Remark 10.1 (What the axiom footprint means mathematically). The audit output—the axioms `propext` and `Quot.sound`—can be read against Carneiro's axiomatization of Lean's type theory [33]. Each named declaration whose axiom footprint is reported in this paper is a theorem of a dependent type theory with inductive types, a hierarchy of universes, and a definitionally proof-irrelevant impredicative universe of propositions, extended by exactly two axioms: propositional extensionality (propositions that imply each other are equal) and the soundness axiom of quotient types. Function extensionality is derivable from these two, so in a mathematician's vocabulary the axioms used amount to: extensionality of propositions, existence of quotients, and, as a consequence, extensionality of functions. In Lean the law of excluded middle is a *theorem* derived from `Classical.choice`, not an independent axiom; not using choice therefore entails that excluded middle is not used as an axiom either.

Carneiro's theorem is stated for Lean 3. For Lean 4 the corresponding metatheory is under active development in the same author's Lean4Lean project [32], which has produced a second typechecker for Lean 4, written in Lean and derived from the C++ kernel implementation rather than independent of it, has formalized parts of the type theory abstractly, and has proved parts of that checker correct; a soundness proof for the Lean 4 kernel has not been completed. The reading that follows therefore transfers a Lean 3 result to a Lean 4 development, on the assumption that the two kernels agree on the fragment used here: inductive types, a hierarchy of universes, a definitionally proof-irrelevant impredicative `Prop`, quotients, and the two axioms reported above. That assumption is plausible and is, to our knowledge, the one the community works under, but this paper does not establish it. Set-theoretically, then, in the model underlying Carneiro's soundness theorem—universes interpreted as V_{κ_n} for inaccessible cardinals κ_n, propositions as the two-valued proof-irrelevant universe $\{\varnothing, \{\bullet\}\}$—the only interpretation clause that references the fixed choice function on the interpreting set is the clause for `Classical.choice`. The interpretation of the theorems audited here therefore does not depend on the selection of a choice function, and their consistency is relative to ZFC together with finitely many inaccessible cardinals. Carneiro's published result is stated for each finite n: the fragment with n universes is interpreted relative to ZFC plus n inaccessibles. A fixed finite development such as this artifact uses only finitely many universe levels, so a concrete finite bound on its universe usage is to be expected; we have not computed one here.

In particular, unconditional extraction of data from a `Prop`-valued existential is exactly what `Classical.choice` licenses; an axiom output without it therefore certifies that the audited dependency terms do not use Lean's generic choice-based eliminator for extracting unrestricted data from an arbitrary `Prop`-valued existence proof. That is the precise sense of the reported footprint; it does not assert that data can never be constructed from a proposition carrying additional constructive structure—the development itself does exactly that, from the strict order. The named declarations are a smaller set than the closure: the shipped logs carry axiom output for 1,655 distinct names, while the closure declares 5,431 in that same unit. (Both forms of the kernel report are counted: the 1,655 include the declarations that depend on no axiom at all, which a scan for the phrase "depends on axioms" alone would silently drop.) What covers the remaining declarations is the token-level source-closure scan of item (2) above, which is a different and weaker instrument. What is *not* claimed: this reading does not establish that the theorems of the development hold in ZF.

Carneiro's soundness proof is itself carried out in classical set theory with choice (ZFC with inaccessibles); what is established is the object-level absence of choice and the independence of the interpretation from the choice function. The ambient type theory also has an impredicative `Prop` and definitional proof irrelevance with uniqueness of identity proofs built in, so it differs both from Bishop's informal practice and from homotopy type theory with univalence, with which UIP is incompatible. The constructive content of the development—moduli, explicit exception sequences, witnesses as data—is carried by the design of the development over presented reals, not by the axiom system.

This remark is complementary to Section 8, and the final observation is a heuristic picture rather than a theorem. Section 8 explains why the general and abstract forms of the statements are not provable in the choice-free setting (in Spitters' sheaf model, sequence avoidance fails); the present remark says inside which axiom system the theorems proved here do hold. Carneiro's construction provides a classical two-valued interpretation of the audited Lean fragment, in which the reals are uncountable and the general forms are true, while the non-classical models on the no-go side explain why unrestricted formulations cannot be expected in weaker constructive settings. The contrast between the two model families makes the shape of the theorem—presented reals, coded profiles, explicit exception sequences—look mathematically motivated rather than stylistic. We emphasize that no published result establishes that Spitters' model interprets Lean's full type theory (impredicative `Prop`, definitional proof irrelevance, quotients), which is why this paragraph is offered as a picture and not as a proof.

10.2 The size of the development

The artifact shipped with this paper is the audited source bundle: the union of the import closures of the six audit roots, 191 development-source Lean files—to which three Lean audit tools under `tools/` are additional—with 83,477 lines and 3,956 declarations by the lexical count defined in Section 10.3. That union is wider than the code the public theorems alone reach: the closure of the public theorem aliases by themselves is 180 of the 191, the remaining eleven being the aggregation module `Mathdemo`, the supplement `SupplementChoiceFreeMeasureDCT`, the reading interface, the Definition 1.1 transcription with its adapter, the two concrete models, the two example modules, and the three axiom-check modules ($1 + 1 + 1 + 1 + 2 + 2 + 3 = 11$). The bundle is to be read against the 49 numbered items (definitions, lemmas, propositions, corollaries, theorems) that the source states from Definition 1.1 through Theorem 4.15.

One remark on the artifact's history belongs here, because the deposit of the previous version is public and the figures differ. Version 0.6.1 carried 513 files; 322 of them declared nothing at all, holding an import, a doc-comment recording a step of the development process, and empty `namespace` blocks. They were relay nodes in a chain up to 169 modules deep, and they inflated every file count in this section while contributing no mathematics. They have been removed and the imports through them rewired to their targets. The effect on the figures is the measure of what they contained: 322 files and 16,593 lines disappear, and the declaration count falls by two—both of them lines inside block comments, which the lexical pattern below counts. No theorem, definition or proof was removed, and the audit output is unchanged.

Two remarks on the counting. A declaration is a source line matching

```
^(private )?(noncomputable )?(theorem|def|lemma|structure|abbrev|instance)
```

so declarations carrying other modifiers, or not starting at the beginning of a line, are missed; and the pattern is purely lexical, so it also matches the occasional line inside a block comment. The figures are the raw counts.

Where the lines are matters more than how many there are, and eighty thousand of them for one theorem invites the question of what they are doing. The answer is visible once they are split by layer.

Layer	Files	Lines	%	Declarations
repository root (public aliases, aggregation, supplement)	3	567	0.7	21
`Mathdemo/` named modules (the proof and the interfaces)	16	42,947	51.4	1,910
`Mathdemo/Internal/Rat` (the rationals)	8	1,157	1.4	130
`Mathdemo/Internal/Real` (the presented reals)	118	13,610	16.3	576
`Mathdemo/Internal/Measure`	1	46	0.1	1
Sec4 convergence layer (`Mathdemo/Internal/Sec4/` plus `Sec4GenIB.lean`)	40	9,810	11.8	443
`Mathdemo/Internal` (not yet grouped)	5	15,340	18.4	875
	191	83,477	100	3,956

One figure in that table is the whole explanation. The presented reals occupy 118 files for 576 declarations: this is the arithmetic and order theory of regular sequences, developed inside the artifact because the reals here are a concrete type rather than an interface satisfied elsewhere (Section 9.2). A development quantifying over an abstract real-number structure does not carry that layer; it refers to it. Against it, the mathematics of the paper – Sections 1–4 of the source, the profile theory, the avoidance, the convergence argument – is the 42,947 lines of the named modules together with the 9,810 of `Sec4`.

The `Measure` directory holds a single file and a single declaration; the measure-theoretic content of the argument is in the named modules and in the Sec4 layer rather than under that path, and the directory name records where an earlier grouping put it rather than where the work is. And the 15,340 lines not yet grouped are the residue of the regrouping described above, which is a statement about the tidiness of the artifact and not about the mathematics.

None of this makes the development small. It makes the size attributable: the layer that dominates is the one the choice of presented reals commits to, and Section 10.3 explains why the closure that contains it is wider than the code the theorems strictly reach.

10.3 What fixes the contents

The contents were fixed by an explicit criterion rather than by a judgment of mathematical importance. The starting point is reachability: seed with every declaration of the modules that carry a claim of this paper—the public aliases of Section 7, the reading interface, the transcription of Definition 1.1 with its adapter, the three concrete models (two of them in modules of their own), and the axiom-check modules—and take the transitive closure of the constants occurring in the type and in the value of each declaration.

That closure is not by itself a tree that compiles, and the gap is not small. The tree the stripping started from held 20,817 declarations. Reachability selects a minority of them, and more have to be added before the type checker accepts the result: material reached through elaboration rather than through a proof term—instances, `simp` lemmas, notation—and declarations that the audit modules name in `#check` and `#print axioms` commands, which are not proof terms at all. The additions were identified by iterating: remove the complement, compile, and restore what the failure identifies—by name where a constant is unknown, and by locating the elaboration-time dependency where the message names only a class to synthesize or nothing at all. The contents are a fixed point of that iteration: a set containing the closure on which the build succeeds, obtained by restoring on failure and nothing else. No minimality among compiling supersets is claimed. The reachability tool reports 52 of the 190 modules as containing no reachable declaration, which invites the obvious test; we ran it. Removing those 52 and rebuilding fails. What the test shows is that the set cannot

be dropped as a set; it does not show that each of the 52 is individually load-bearing, and we do not claim that. Material reached through elaboration rather than through a proof term—instances, `simp` lemmas, notation—is invisible to the reachability tool and visible to the type checker, which is the gap the count measures. In the tree as shipped the split is 2,552 declarations reachable from the claim-carrying modules against 2,879 not reachable, 5,431 in all; these three figures are produced by `tools/reachable_core.lean`, whose output is shipped as `logs/reachable_core.txt`. The reachable ones are connected to a claim of this paper by construction. For the rest, the description above is of the procedure the author followed, not of a property the shipped evidence certifies: the iteration logs are not shipped, and what the machine evidence checks is the final build and the final reachability counts.

No formal-to-informal ratio is reported for the result; Wiedijk's de Bruijn factor is a ratio of file sizes measured on a finished formalization [34], and the counts here serve to say what the artifact contains, not what it cost. The counts of the last paragraph are taken in the elaborated environment—constants carrying a source position, the unit the reachability tool works in—which is finer than the lexical pattern of the previous subsection: by it the shipped tree holds 5,431 declarations, against the 3,956 lines that the pattern matches. The two units are not interchangeable, and no figure below mixes them.

Being connected is weaker than being needed, so the result is not claimed to be minimal: a proof term can mention a lemma the proof would go through without, and reachability keeps it. What is claimed is checked. Two invariants are machine-checked and shipped: the elaborated type and the kernel axiom footprint of each of the 21 public aliases (`tools/public_surface_check.lean`, recorded in `logs/public_surface.txt`), and the presence of every identifier this paper cites by name (`tools/check_frozen_names.sh`, against the 118 entries of `tools/frozen_names.txt`, of which the 98 that name declarations are checked by asking Lean to elaborate them, and the two that name hypothesis binders—`hconv` and `hdom` above—by locating them in the signature they belong to). Both hold of the shipped tree, and both are the invariants any later reorganization of the artifact must preserve.

10.4 What the bulk of the development consists of

The formal-to-informal difference is not uniform overhead; at least three kinds are mixed. First, notational overhead: rewritings that a person performs in one line, placed as explicit lemmas. This part is genuinely redundant. Second, material the source presupposes: the foundational layer, which is not a difference from the source but a difference of reference target. Third, constructive content that the source suppresses: moduli of convergence, domain management for partial functions, apartness data. Where Bishop and Cheng write "the series converges", the formalization must carry the rate. Only the third kind is mathematically interesting, and separating the three cleanly remains open. The second kind, however, can be decomposed against the reference itself.

Compared clause by clause with Chapter 2 of *Foundations of Constructive Analysis*, the foundational layer consists of a transcription of that chapter—regular sequences and Bishop equality with the carrier left unquotiented, the standard bound, the index-shifted arithmetic operations and their laws, the one-point positivity condition of its Definition 3 and the tail form of its Lemma 2, the two orders, and limits with moduli—together with five kinds of newly paid cost: (i) the construction of the rational layer itself, as a decidable ordered field, which is the reference of the reference; (ii) the substitution of the dyadic gauge 2^{-k} for the $1/n$ gauge, translated in both directions by the reindexing $x'_k := x_{2^k}$ (inverse $k(n) := \lceil \log_2 n \rceil$), with every margin constant redesigned under the new gauge; (iii) the discharge, at every operation and predicate, of respect for Bishop equality—the *Foundations* disposes of this in a single remark, that the map from a real to its n-th

rational approximation "is not a function"; (iv) the carrying and recovery of existential witnesses—the *Foundations* declares once that in all its existence proofs the witness is understood to have been extracted, and that for positivity "the n was implicitly there all along"; Appendix B.12 is the machine-checked statement that this looseness is harmless without choice; and (v) an explicit modulus for every convergence statement. In one sentence: *most of the foundational stratum is the cost of discharging, at every point of use, conventions that Bishop declared once in prose—witnesses count as extracted, representation-level operations are not functions, a real is not an equivalence class.*

The comparison with Séméria closes the accounting. His CoRN development quantifies over an abstract `ConstructiveReals` interface whose implementations and theory live in the Coq standard library—the subject of his JFLA paper [27]—so the counterpart of that layer sits *outside* his artifact, while `Print Assumptions` still descends into it and reports no axioms. Bishop and Cheng refer to *Foundations*; Séméria refers to the standard library; the present development carries its own. That layer is the cost of internalizing the reference. (These three are all on the representation side of the divide; the fourth coordinate, reaching the same proposition/data tension from the extensional side, is Booij's locator theory—Appendix B.13.)

10.5 Limitations and validation boundary

The result should be read with the following limitations in view.

(1) **Integration-space fidelity.** Definition 2.1 differs from the source's Definition 1.1 by the absence of the inhabitedness and normalization requirements—a pure weakening, in the direction that widens the range of application: (Remark 2.2) every integration space in the sense of the displayed Definition 1.1 satisfies Definition 2.1, and the theorems apply to it. The adapter from the clause-by-clause encoded transcription is hypothesis-free, and `Mathdemo.SourceIntegrationSpaceDef11` derives all fourteen fields of Definition 2.1 from Definition 1.1 (Remark 2.2). We note also that the field `I_nonneg` of Definition 2.1 is derivable from the continuity clause, a derivation machine-checked in the artifact, so that carrying it as a field is unnecessary.

(2) **Concrete semantics.** Three concrete examples are shipped: the degenerate zero-integral model on `PUnit`, which verifies that the public wrapper can be applied end to end; the normalized point-evaluation model `Mathdemo.DiracIntegrationSpace` (Remark 2.2); and the finitely supported model `Mathdemo.FiniteWeightedIntegrationSpace`, whose instance at two points is unconditional and whose closed instance on `Bool` is proved to give both singletons positive measure. The integral there is the unscaled sum over the listed points, so the multiplicities it realizes are the integers obtained by repeating a point, not arbitrary positive reals, and no model over a continuum is constructed. What the three certify is that the interface is satisfiable, that normalization is realizable, that the adapter applies unconditionally, and that the axioms admit a model in which the measure is not concentrated at a single point; they are not offered as a survey of expressive range.

(3) **Hypothesis on the limit.** The public theorem assumes that the limit f is already an integrable representative. It proves convergence of the integrals under that hypothesis; it does not derive integrability of a merely measurable limit.

(4) **Logical and real-number scope.** The development uses Lean's impredicative `Prop` and regular-sequence presented reals. It is neither a predicative reconstruction nor a theorem

over arbitrary constructive real-number objects, and it does not overturn the model-theoretic no-go results discussed in Section 8.

(5) **Comparison and priority.** Séméria's CoRN development already contains a Bishop–Cheng dominated-convergence theorem, and the two theorem surfaces live over different real-number and integration-space interfaces. No claim of priority or strict formal generalization is made.

(6) **Implementation maturity.** The artifact is fixed by reachability from the public claims (Section 10.3), and its modules are grouped by layer (`Mathdemo/Internal/Rat`, `Real`, `Measure`, `Sec4`). That criterion keeps what the theorems depend on; it does not produce a library with a designed interface, and the implementation modules beneath the public surface are not a public API. The public aliases and principal proof route have been reviewed and audited, while detailed manual review of every low-level internal declaration remains ongoing. A lexical scan of the closure finds no declaration whose conclusion is written, on one line, in the form `: True :=`. The scan is lexical and reaches declarations, not the fields of a structure instance: one field assignment of the form `f := True` survives, at `Mathdemo/Internal/Real/ClosingStrictUpperTransferBridge.lean`, in a structure whose substantive field is genuinely proved and whose `True`-valued field no declaration reads. Five were present through v0.4.1 and were removed in v0.4.2: `ContinuousOn`, `lemma33_lt_of_not_le`, `lemma34_out_exists_cell`, `fatou_type_stub_not_source_4_14`, and—the one whose name most invites misreading—`thm_4_15_dominated_convergence`, a `def` carrying the hypotheses of dominated convergence and concluding `True`. All five were vestiges of earlier generic routes, none was referenced anywhere in the closure, and a declaration whose conclusion is `True` carries no information, so none could contribute to any proof; the theorem of this paper is `bishop_cheng_thm_4_15_propC`, which is unrelated to that vestige and is stated and proved in full. They are recorded here because at the time they were present the token-level source-closure scan of Section 10.1, item (2), detected vacuous *proofs* (`sorry`, `admit`) and not vacuous *statements*. That scan now also rejects a conclusion of `True`, which is the form all five took. It is a lexical guard: it matches the pattern `: True :=` on a single line, so a vacuous statement whose type is written across lines, or whose conclusion merely reduces to `True`, escapes it. The complete check is therefore carried out separately, by `tools/vacuous_statement_check.lean`: it enumerates the 5,431 declarations the development writes in source, sets aside the 509 that are not statements—inductive types, constructors, recursors and quotient primitives—and for each of the remaining 4,922 strips the leading binders from the type, reduces the conclusion to weak head normal form, and reports it if that conclusion is `True`. It finds eight, and they are of one shape: a membership lemma whose ambient set is `Set.univ`, so that $x \in S$ unfolds to `True` and the proof is `Set.mem_univ`. None is a vestige—each is used, one to four times, to supply the membership proof term an interface demands, and the interface is the one a model with a smaller domain would have to satisfy non-trivially. What is vacuous is the proof obligation in these particular constructions, not the role the statement plays. The eight are recorded in `tools/vacuous_statements_known.txt` with that reason, and the check compares its findings against that list in both directions, so a new vacuous statement fails the audit and a name that stops being vacuous cannot leave the list stale. The output is shipped as `logs/vacuous_statements.txt`. Its range is the two library roots rather than the six roots of the source scan, because it reads an elaborated environment and `SupplementChoiceFreeMeasureDCT.lean` is not a library target: it is a file of `#check` and `#print axioms` commands, run by `lake env lean`, and it declares nothing for this check to examine.

A complete `./build_audit.sh` run from the v0.7.2 tree finished on 2026-09-05 with `BUILD_AUDIT_EXIT=0`. The shipped log carries the banner `artifact_version=0.7.2` and records the successful build stages (2534, 2535, 2536, 3 and 2548 jobs), the declaration-level axiom checks reporting `[propext, Quot.sound]`, the reading-layer axiom check over 104 declarations, and the static audit traversing its closure within the same run. Seven reference logs are shipped, archived from that run rather than written by it: the script itself writes to the `.rerun.txt` files, and the references are promoted from them once, when a release is cut. Six come from `build_audit.sh`—`logs/build_audit.txt`, `logs/static_audit.txt`, `logs/mathdemo_build.txt`, `logs/reading_layer_axioms.txt`, `logs/public_surface.txt` and `logs/vacuous_statements.txt`—and the seventh, `logs/reachable_core.txt`, is the output of `tools/reachable_core.lean`, which is run separately. None of them names a path from before the implementation modules were regrouped into `Mathdemo/Internal/Rat`, `Real`, `Measure` and `Sec4`.

The scope of the build audit should be delimited. The four reading-layer modules—`Mathdemo.MathematicalInterface`, `Mathdemo.SourceIntegrationSpaceDef11`, `Mathdemo.DiracIntegrationSpace` and `Mathdemo.FiniteWeightedIntegrationSpace`—are not in the dependency cone of `Mathdemo.CheckSec3PortAxioms`: they are a layer for *reading* the development, not a layer the development *depends on*, so they are not built in the initial theorem-root stages; they are built by the later aggregate stage. They are audited by two other routes. First, the static source closure of Section 10.1 covers them, since the audit roots include the aggregation module. Second, a separate aggregated build, `lake build Mathdemo`, completes, and a dedicated reading-layer audit covers the reading-layer declarations, which are outside the dependency cone of the main theorem and are therefore not reached by the first route; the full inventory and its split are given in Section 10.6, item (3). The aggregate build is part of `build_audit.sh`; the reading-layer axiom audit is run from a check file outside the tracked closure, so that the audited closure is unchanged by the check itself, and its output is shipped (Section 10.6).

This tree is `v0.7.2`, tagged in the public repository and deposited with the version DOI `10.5281/zenodo.22309608`. Both were made from the exact tree audited above, after the checksums were regenerated on it. The requirement that the released tree receive a complete `./build_audit.sh` run is met by the run just described. The software deposit [39], concept DOI `10.5281/zenodo.21850965`, which resolves to the latest version, carries the Lean code and the audit apparatus only; the paper source is excluded from it by `.gitattributes` and is not part of the deposit.

10.6 Artifact interface and reproducibility

The paper cites the public aliases in `ChoiceFreeMeasureDCTPublic.lean`. That file is the whole of the intended interface: 21 declarations in the namespace `ChoiceFreeMeasureDCT`, each carrying a doc-string that states what it is. The supplementary file `SupplementChoiceFreeMeasureDCT.lean` checks and prints axioms for both the aliases and their implementation declarations. The artifact is [39].

A reader can inspect the result at three levels. They differ in what they are capable of detecting, and it is worth naming that before the commands: *integrity* compares bytes against the shipped checksums and detects a changed file, but nothing about whether the code proves anything; *re-execution* runs the kernel and the static scan and detects a broken proof, but only for the tree in front of it; and *release consistency* compares the paper, this artifact's three metadata files and the logs, and detects a figure or a version that has drifted while everything else still passes. A reader chasing a specific doubt should start at the level that can see it.

(1) **Public surface.** Begin with the following two files:

`ChoiceFreeMeasureDCTPublic.lean`
`Mathdemo/ChoiceFreeDCTExamples.lean`.

They display the callable interfaces without requiring navigation through the implementation modules.

(2) **Short checks.** Run the static audit, the root checksum command, and then the independent checksum command inside `audits/corn/`:

```
python3 tools/static_no_choice_audit.py
sha256sum -c SHA256SUMS
(cd audits/corn && sha256sum -c SHA256SUMS)
```

These checks do not rebuild Lean but verify the public source closure and the integrity of the shipped evidence.

(3) **Complete Lean verification.** Run `./build_audit.sh`. The script builds the principal theorem root, public support modules, the concrete example, and the public alias files; invokes the dedicated axiom checks; and finishes with the strengthened static audit. A successful run ends with `BUILD_AUDIT_EXIT=0`. The script also performs the aggregate `lake build Mathdemo`, the second of the two audit routes described in Section 10.5, whose record is shipped as `logs/mathdemo_build.txt`; it then generates a temporary reading-layer check file outside the tracked closure, runs `lake env lean` on it to print the axiom footprint of all 104 reading-layer declarations—the facade's 36, the transcription's 27 including the hypothesis-free adapter and the gauge bridge, the point-evaluation model's 10 and the finitely supported model's 31—and verifies that 102 report `[propext, Quot.sound]` and that the two which report no axioms at all are `BishopCheng.ComplementedSet` and `BishopCheng.boolPts`, shipping the output as `logs/reading_layer_axioms.txt`.

Local reruns are written to:

`logs/build_audit.rerun.txt`
`logs/static_audit.rerun.txt`
`logs/reading_layer_axioms.rerun.txt`
`logs/public_surface.rerun.txt`
`logs/mathdemo_build.rerun.txt`
`logs/vacuous_statements.rerun.txt`.

Every log the audit writes goes to one of these six, so the shipped reference logs stay byte-identical and checksum verification remains meaningful after rerunning the audit; the reference logs are promoted from the rerun files once, when a release is cut, and the checksums are generated after that promotion. The independently packaged CoRN reproduction is documented in `audits/corn/CORN_ASSUMPTIONS_AUDIT.md`; it can be checked without trusting the Lean build or the comparison prose in this paper.

11 Conclusion

The artifact realizes a complete profile-based route from convergence in measure and full-set domination to convergence of integrals over presented reals, relative to the explicit integration-space interface of Definition 2.1. Its central constructive step is not the suppression of the exceptional levels returned by profile theory, but their explicit enumeration followed by a presented-real countable-avoidance construction. That distinction explains both how the formal proof proceeds and why it does not contradict the known obstruction for unrestricted constructive real-number objects.

The result is exposed through proof-relevant global and full-set interfaces, automatically synthesized wrappers, and a Prop-facing interface, and the final public declarations are accompanied by kernel dependency output, a transitive source audit, checksums, and a pinned reproduction of the closest CoRN prior work. The limitations are equally explicit: the limit is assumed integrable; the shipped concrete models reach finitely supported measures with integer multiplicities, and no model over a continuum is constructed; the integration-space interface is a clause-level strict weakening of the encoded Bishop–Cheng Definition 1.1, the omissions being inhabitedness and normalization (Remark 2.2); and the development is not predicative. These boundaries leave clear next tasks while keeping the present theorem and its audit precisely scoped.

Statements and Declarations

Funding. No funding was received for conducting this study.

Competing interests. The author has no competing interests to declare that are relevant to the content of this article.

Author contributions. The paper has a single author, who is responsible for all of it.

Data and code availability. There is no dataset. The Lean 4 artifact, the audit apparatus and the shipped audit logs are archived and citable at `10.5281/zenodo.22309608`, and are also at `https://github.com/turahigashi/choicefree-bc-dct` under tag `v0.7.2`, which is the commit the deposit was cut from. Every figure this paper states about the artifact is produced by a run recorded in the deposit, and the deposit contains the scripts that regenerate them: `./build_audit.sh` rebuilds the development and re-runs the checks of Section 10 that need only Lean and Python, ending in `BUILD_AUDIT_EXIT=0`. Three of the reported figures are outside it and are reproduced separately: the reachability counts by `tools/reachable_core.lean`, the root checksum verification by `sha256sum -c SHA256SUMS`, and the CoRN audit by the commands packaged under `audits/corn/`, which need Rocq. The checksums shipped with the deposit verify that the bytes audited are the bytes distributed. Reproducing the build requires Lean 4.30.0 and mathlib 4.30.0; the remaining checks need only Python 3 and `coreutils`. The reproduced Rocq audit of the CoRN development discussed in Section 9 is packaged under `audits/corn/` with its own checksum manifest, and pins both the source commit and the Rocq version.

Ethics approval. Not applicable.

Use of AI-assisted tools

Generative AI and AI-assisted coding tools were used extensively in the development, refactoring, documentation, and review of the Lean source code, and in drafting and revising the prose of this paper. The author determined the mathematical scope and formalization goals, reviewed the public theorem statements and proof architecture, reviewed and approved the manuscript text, and executed the reported build, static, checksum, and axiom audits. AI systems are not listed as authors, and the author assumes full responsibility for the contents of the paper and artifact.

That disclosure bears on the claims of this paper unevenly, and the difference is worth stating rather than leaving to the reader. Every theorem reported here is checked by the Lean kernel; the axiom footprints of Section 10.1 are the kernel's own output, and the audits that produce them are scripted and reproducible from the deposit. Whether a proof term was written with AI assistance has no bearing on whether it type-checks. In this respect the reported results do not rest on the reliability of the tools used to produce them, and a reader who distrusts those tools entirely can still run `./build_audit.sh` and obtain the same output.

What a kernel cannot check is whether the statements are the right statements, and that is where AI assistance could mislead. Four judgements in this paper are of that kind: whether Definition 2.1 renders Bishop and Cheng's Definition 1.1 faithfully (Remark 2.2), whether the displayed Lean definitions unfold to the definitions of Part I (Section 7.1), whether the novelty claims of Section 9 are correct, and whether the prior work is characterized accurately. The paper marks each of these as a judgement at the point where it is made, rather than presenting it alongside the machine-checked material. Line-by-line human review of the whole implementation, as distinct from the review of the public statements and the proof architecture described above, has not been completed; the separation just drawn is the reason the reported results do not depend on its completion.

A From the mathematics to the Lean declarations

The tables in this appendix let a reader move between Part I, the source, and the artifact. They are the only place in which the correspondence is asserted; the statements in Part I are self-contained mathematics, and the declarations in Part II are self-contained Lean.

A.1 Numbering: this paper and the source

This paper numbers items by section, so its numbers do not coincide with those of Bishop and Cheng's memoir; since both schemes contain items numbered "3. x" and "4. x", the correspondence is collected here. The headings in Part I also carry the source numbers.

Notes in the last column assert agreement of mathematical content with the memoir; they do not assert word-for-word agreement with its English text.

This paper	Content	Source	Note
Definition 2.1	integration space (working form)	Definition 1.1	clause-level strict weakening of Definition 1.1: all fourteen fields derived, hypothesis-free, from the transcription at the memoir's own gauge, with partial functions and the integral on L as the memoir has them (Remark 2.2; Appendix B.6)
Definition 3.1	full set	Definition 1.9	same statement
Definition 3.2	complemented set	Definitions 2.1, 2.2(d)(e), 2.3	same core statement; merges $-A$, $A - B$, integrability, measure
Definition 3.3	convergence in measure	Definition 4.11	same statement
Definition 3.4	domination on full sets	§1 notational convention	the convention with the quantifiers displayed; index-dependent full sets already licensed by it—not a weakening
Definition 3.5	profile	Definition 3.1	stated by the formalization in coded form (Appendix B.4); no continuity clause—a weaker hypothesis, hence more general
Definition 3.6	smooth level	—	introduced in the source inside the text of its Theorem 3.5
Theorem 4.1	dominated convergence	Theorem 4.15	same assertion; hypotheses phrased differently (Remark 4.3, Section 6)
Theorem 5.1	smoothness away from an explicit sequence	Theorem 3.5	the formalized statement returns the explicit family constructed by the source's proof (Appendix B.5); the tail conditions are quantified over $[a, b]$, as the source's $F \subseteq C[a, b]$ has them; Part I keeps the source's wording—not a strengthening (Appendix B.5)
Theorem 5.2	integrability of level sets	Theorem 3.6	A pair and B pair constructed independently (Appendix B.8); exported in the source's global form, one exceptional sequence for all $t > 0$, obtained as the source obtains it, by covering $\mathbb{R}^+$ with the intervals $((n{+}2)^{-1}, n{+}2)$ and flattening the per-interval sequences
Lemma 5.3	countable avoidance	—	not a separate stage in the source; contained in the transition from its Theorem 3.5 to its Theorem 3.6 (Appendix B.3)
Proposition 5.5	uniform complement estimate	hypotheses of Lemma 4.14 + construction of Theorem 4.13	returns (A, N, δ) as data; reparameterization $B := -C$ (Appendix B.10)
Proof step (4), Section 6	two-part estimate	Lemma 4.14	same two-part estimate
Appendix B	Lemma 1.2, Corollary 1.3, Definition 7.1, and others	same numbers	the appendix retraces the source's arguments and cites the source's numbers directly

A.2 From mathematical objects to Lean declarations

Axiom of Definition 2.1	`IntSpaceC` field
(A1) respect for Bishop equality	`L_resp`, `I_resp`
(A2) closure of L	`add_mem`, `smul_mem`, `abs_mem`, `cutPos_mem`
(A3) linearity of I	`I_add`, `I_smul`
(A4) positivity of I	`I_nonneg`
(A5) truncation limits	`cutNat_tendsto_raw`, `cutSmall_tendsto_raw`
(A6) constructive continuity	`continuity`

Mathematical object	Public alias	Implementation
Definition 3.3	—	`ConvergeInMeasureC`
Definition 3.4	—	`DominatedOnFullC`
Theorem 4.1	`bishop_cheng_dominated_convergence_propC`	`bishop_cheng_thm_4_15_propC`
Theorem 5.1	`profile_smooth_away_from_sequenceC`	`thm_3_5_smooth_aeC`
Lemma 3.4 (partition data)	`profile_partition_dataC`	`lemma_3_4DataC`
Theorem 5.2	`profile_level_sets_integrable_apart_globalC`	`thm_3_6_all_posC`
Theorem 3.6, interval-local core	`profile_level_sets_integrable_apartC`	—

Lemma 5.3	—	`Thm36B1ApartPointDataC`
		`thm36B1_apartPointDataC`
		`thm36B_smoothPointDataC_construct`

Proposition 5.5	`uniform_complement_from_profile_levelsC`	`lemma43UniformComplementData_of_majorantC` `lemma43UniformComplementData_of_majorantOnFullDataC`
Proof step (2)	—	`lemma43DyadicSmoothDataC_construct`
Proof step (4)	—	`lemma_4_14_local_two_epsilonC`
Proof step (5)	—	`integral_diff_lt_of_abs_error_integral_ltC`

Public aliases are in the namespace `ChoiceFreeMeasureDCT`; implementation declarations are in `BishopSec1P` or `BishopSec3P`. The complete list of public aliases, including the Type-valued route and the automatic wrappers of Section 7, is in `ChoiceFreeMeasureDCTPublic.lean`.

B Constructions specific to the formalization, and their mathematical status

In the spirit of the blueprint genre, but in the reverse direction: these sections state, as ordinary mathematics, what the formal development had to construct. The appendix has two purposes. The first is to write down, without Lean vocabulary, the constructions that the development did *not* receive from Bishop and Cheng but had to build in the course of the formalization. The second is to make explicit which parts of that work are routine, which are known, and which were the actual difficulties (Appendix B.11). Part I suffices for the mathematical content; this appendix discloses, in mathematical language, what the formalization concretely consisted of.

Throughout the appendix, presented reals are taken in the dyadic-gauge form used by the implementation: regularity reads $|x_m - x_n| \le 2^{-m} + 2^{-n}$. Section 2.1 stated the $1/n$ form of the

Foundations; the two are translated by an explicit reindexing, and the translation is part of the cost accounted in Section 10.4.

B.1 The proof flow against the source

The conclusion first: the outline of the formal proof coincides with the source's. The number of stages and their order are the same, and the differences concentrate in how each stage is *stated*. The single exception is countable avoidance, which in the source is not a separate stage at all: it lives inside the two transition sentences at the beginning of the proof of its Theorem 3.6, immediately after Theorem 3.5 is invoked ("all but countably many points t of (a, b) are smooth"; "let t be smooth"). Constructively that transition cannot be executed as written, and the development had to erect it as a theorem of its own (Appendix B.3).

The dependency structure of the proof, with the status of each stage relative to the source, is the following; arrows indicate constructed data or a theorem application, as in Section 6.

- Clauses (1)–(4) of Definition 1.1 ⇒ Lemma 1.2 ⇒ Corollary 1.3 at $k = 1$ ⇒ positivity of I. Status: as in the source; that positivity is derivable, so that carrying it as an axiom is unnecessary, is an observation of this development (Appendix B.6).

- Coded profile and the partition data of Lemma 3.4 ⇒ Theorem 3.5 (smoothness) together with the explicit exception sequence. Status: statements as in the source; the coded form and the sequence-returning form are formalization-specific (Appendices B.4, B.5).

- Presented-real order with cotransitivity as data ⇒ countable avoidance by cut points fixed in advance at a margin. Status: the statement is Bishop's, and so is the comparison against a separated pair; what the formalization supplies is the margin, fixed before the point is read (Appendices B.2, B.3).

- Avoidance applied to the exception sequence ⇒ apart thresholds ⇒ Theorem 3.6: the A pair and the B pair constructed independently, with the measure equality; the signed double series over a fixed shell enumeration is consumed here (Appendices B.8, B.9).

- Dyadic levels $\alpha_n \in (2^{-(n+1)}, 2^{-n})$ as data (source Lemma 4.3) ⇒ uniform complement estimate returning (A, N, δ) (source: hypotheses of Lemma 4.14 and the construction inside Theorem 4.13) ⇒ two-part estimate (Lemma 4.14) ⇒ Theorem 4.1 = source Theorem 4.15, with full-set domination and the majorant $H = |g| + |f|$ (Appendix B.10).

The differences concentrate at four places.

(a) From existence to data. Where the source writes "there exists", the development frequently writes a function returning the object: clauses (2) and (4) of Definition 1.1, the level sequence of Lemma 4.3, the triple (A, N, δ) in the hypotheses of Lemma 4.14. Under Bishop's intended BHK reading of $\exists$ this is faithfulness rather than strengthening; as a Lean interface, however, it is a data refinement of the corresponding `Prop`-valued statement (Section 7.2) (Appendix B.12).

(b) From "countably many" to an explicit sequence. Theorem 3.5's "all but countably many" cannot be handed to the avoidance construction as stated; only the form that returns an explicit sequence can be (Appendix B.5).

(c) From an appeal to a separated pair to a margin fixed in advance. Inside the avoidance, a split at a midpoint is undecidable, and Bishop compares against a separated pair instead. A margin is fixed before the point is read, the interval is cut at the two points it determines, and the middle is discarded (Appendix B.3).

(d) The B pair does not follow from the A pair. Where the source dismisses the B side as essentially similar, the fact that $\leq$ is $\neg <$ forces an independent construction, with the monotonicity and the usable implications reversed (Appendix B.8).

None of the four alleges a mathematical error in the source; they are obligations exposed when its argument is executed under the present choice-free discipline, and its omissions are the usual "essentially similar" economies of print. The four points say what is missing if one tries to execute the text directly as a construction, and the sections below supply exactly that.

B.2 The order on presented reals as data

Definition B.1 (Presented real, dyadic form)**.** A *presented real* is a sequence $x = (x_n)_{n\geq 0}$ of rationals such that

$$|x_m - x_n| \;\leq\; 2^{-m} + 2^{-n} \qquad (\forall m, n)$$

(a *regular sequence*). Two presented reals x, y are *equal* ($x \approx y$) if

$$\forall k\; \exists N\; \forall n \geq N: \quad |x_n - y_n| \leq 2^{-k}.$$

A real is the regular sequence itself; no equivalence classes are taken. Below, $\mathbb{R}$ denotes the presented reals in this sense.

Definition B.2 (Order, and its data form)**.** $x < y$ means that there exist k, N with

$$\forall n \geq N: \quad y_n - x_n \;>\; 2^{-k}$$

(the development takes the gauge $2^{-(k+1)}$; nothing below depends on that choice). $x \leq y$ means $\neg\,(y < x)$, and x and y are *apart* when $0 < |x - y|$.

Data form: evidence for $x < y$ is a pair (k, N) together with the proof of its property. "To have $x < y$ as data" means holding this pair, as distinguished from the mere truth of $x < y$. The distinction runs through the entire appendix.

Fact B.3 (Cotransitivity—the constructive pivot)**.** *Suppose $x < y$ is held* as data*. Then for every $z \in \mathbb{R}$ one can* decide *which of*

$$x < z \qquad \textit{or} \qquad z < y$$

holds, obtaining evidence for the decided side.

Reason. *The evidence (k, N) gives $y_n - x_n > 2^{-k}$ for $n \geq N$. Fix any $n_0 \geq \max(N, k+4)$ and compare the rational z_{n_0} with $\frac{x_{n_0}+y_{n_0}}{2}$. Rational comparison is decidable, and the comparison selects one of the two cases. At n_0 the gap from the midpoint to either endpoint exceeds $2^{-(k+1)}$. Passing to a later index $n \geq n_0$ moves* both *members of the selected difference, so the regular-sequence error to subtract is $2(2^{-n} + 2^{-n_0}) \leq 4 \cdot 2^{-n_0} \leq 2^{-(k+2)}$; this still leaves the positive margin $2^{-(k+2)}$, so the selected side actually holds (the regularity error of the running index is absorbed by the eventual quantification, and the implementation realizes this extra unit of precision through the shifted subtraction index; the general mechanism is Appendix B.12).* □

The essential point is that *a gap is needed.* If $x < y$ were not held—if x and y might touch—the decision could not be made: for presented reals, the one-point dichotomy "$z \leq m$ or $m \leq z$" is undecidable.

B.3 Countable avoidance by a two-point cut with a margin

This is the most characteristic construction of the development. The goal is the following theorem, which is exactly the assertion of Bishop's well-known nested-interval method—the *statement* is known, and so is the method. Carrying it out as a construction over presented reals, however, cannot use a midpoint split, and needs the comparison points to be fixed in advance.

Theorem B.4 (Countable avoidance; = Lemma 5.3)**.** *Suppose $a < b$ is held as data, and let $(s_j)_{j\geq 0}$ be an arbitrary sequence of presented reals. Then one can* construct *$t \in (a, b)$ apart from every s_j:*

$$a < t < b, \qquad 0 < |t - s_j| \quad (\forall j).$$

No choice principle is used.

Why a midpoint split fails. A classical argument takes the midpoint $m = \frac{l+r}{2}$ of the current interval $[l, r]$ and keeps the right half if $s_j \leq m$, the left half otherwise. But as noted after Fact B.3, the position of s_j relative to the *single* point m cannot be decided, so this branch cannot be constructed.

Comparing against two separated points: Bishop's step. Bishop's own proof of the theorem does not split at a midpoint. It compares s_j against a *separated pair*, the branch being furnished by the corollary to Proposition 7 of the same chapter [1, Ch. 2], and keeps a subinterval clear of s_j. That is Fact B.3 applied as it stands, and it is the step this construction follows; nothing below is a departure from it.

What the formalization adds: a margin fixed in advance. What has to be supplied to carry the step out as a construction is the *schedule*: the two points must be produced before s_j is looked at, with a separation known in advance, so that the branch is a function of rational approximations rather than of an appeal to the pair's existence. That is the margin below, and the width estimate that follows from it.

Definition B.5 (Margin and cut points)**.** For $d \in \mathbb{R}$ set the *margin*

$$\sigma(d) \;:=\; \tfrac{1}{4}\, d$$

(the development implements it by halving twice). What is required of σ is exactly two properties: $0 < \sigma(w)$ and $2\sigma(w) < w$; the choice $\sigma(w) = \frac{1}{4}w$ is merely the simplest that satisfies them. For an interval $[l, r]$ with $l < r$ and width $w := r - l$, the *cut points* are

$$q_L \;:=\; l + \sigma(w), \qquad q_R \;:=\; r - \sigma(w).$$

Lemma B.6. *If $l < r$ is held as data, then*

$$l \;<\; q_L \;<\; q_R \;<\; r, \qquad q_R - q_L = \tfrac{1}{2}\, w,$$

and in particular $q_L < q_R$ is held as data.

Proof. *From $w > 0$ we get $\sigma(w) = \frac{1}{4}w > 0$, hence $l < l + \sigma(w) = q_L$ and symmetrically $q_R < r$. Moreover*

$$q_R - q_L = (r - \sigma(w)) - (l + \sigma(w)) = w - 2\sigma(w) = w - \tfrac{1}{2}w = \tfrac{1}{2}w > 0. \qquad \square$$

Definition B.7 (Interval data)**.** Fix $a < b$ and the sequence (s_j). *Interval data at stage n* is a pair (l, r) of presented reals satisfying, with evidence, the four conditions: (i) $l < r$; (ii) $a < l$ and $r < b$; (iii) $r - l \leq (b - a)\, 2^{-n}$; (iv) for every $j < n$: $s_j < l$ or $r < s_j$. Condition (iv) is read "s_j is *avoided*".

Lemma B.8 (One step)**.** *From interval data $I = (l, r)$ at stage n one can* construct *interval data $J = (l', r')$ at stage $n + 1$ with $l \le l'$ and $r' \le r$.*

Proof. *Let $w := r - l$ and take q_L, q_R as in Definition B.5. By Lemma B.6, $q_L < q_R$ is held as data. Apply Fact B.3 to this pair and $z := s_n$, and distinguish the two cases it returns.*

Case 1: $q_L < s_n$. *Put $J := (l, q_L)$. (i) $l < q_L$ is Lemma B.6. (ii) $a < l$ by hypothesis, and $q_L < r < b$. (iii) The width is*

$$q_L - l = \sigma(w) = \tfrac{1}{4}w \;\le\; \tfrac{1}{4}\,(b-a)2^{-n} \;\le\; (b-a)2^{-(n+1)}.$$

(iv) For $j < n$, avoidance is preserved because $J \subseteq I$ ($s_j < l = l'$ or $r' \le r < s_j$). For $j = n$, the case hypothesis $q_L < s_n$, i.e. $r' < s_n$, is precisely the avoidance.

Case 2: $s_n < q_R$. *Put $J := (q_R, r)$. Conditions (i)–(iii) follow symmetrically, the width being $r - q_R = \sigma(w)$. Condition (iv) for $j < n$ is as before, and for $j = n$ the case hypothesis $s_n < q_R = l'$ is the avoidance.* □

Remark B.9 (Discarding the middle). The surviving interval has $\frac{1}{4}$ of the original width, smaller than the $\frac{1}{2}$ a midpoint split would give. This is not a benefit of faster convergence; it is the consequence of being forced to *abandon the central half*. The classical argument exhausts the interval by the two-way split "left or right"; constructively that split is undecidable, and only after surrendering the region between the two cut points does a decidable two-way split appear. This operation—replacing an undecidable dichotomy by a decidable one purchased with a margin—recurs throughout the development (Appendix B.12).

Proof of Theorem B.4. For the initial interval, apply the cut of Definition B.5 to (a, b) itself:

$$I_0 \;:=\; \big(a + \sigma(b-a),\; b - \sigma(b-a)\big)$$

(the artifact makes the same choice). By Lemma B.6, $a <$ left endpoint $<$ right endpoint $< b$, and the width is $\frac{1}{2}(b-a) \le (b-a)\cdot 2^0$, so condition (iii) holds at $n = 0$; condition (iv) at $n = 0$ is vacuous. Iterating Lemma B.8 yields interval data $I_n = (l_n, r_n)$ for every n. By construction (l_n) is nondecreasing, (r_n) is nonincreasing, and

$$0 \;<\; r_n - l_n \;\le\; (b-a)\,2^{-n} \;\longrightarrow\; 0.$$

For $m \ge n$ we have $l_n \le l_m \le r_m \le r_n$, hence $|l_m - l_n| \le (b-a)2^{-n}$: the left endpoints form a Cauchy sequence with an *explicit modulus*. The coefficient $b - a$ is itself a presented real; taking a rational Archimedean bound $b - a \le K$, accuracy 2^{-k} is reached from index $n \ge k + \lceil \log_2 K \rceil$ on, a numerical modulus. Completeness of the presented reals (taking a diagonal sequence) constructs the limit $t := \lim_n l_n$, with $l_n \le t \le r_n$ for all n.

From $a < l_0 \le t$ and $t \le r_0 < b$ we get $a < t < b$ (composing a strict with a weak inequality is the standard transitivity lemma, itself a consequence of cotransitivity). For each j, condition (iv) of I_{j+1} gives $s_j < l_{j+1}$ or $r_{j+1} < s_j$; combined with $l_{j+1} \le t \le r_{j+1}$ this yields $s_j < t$ or $t < s_j$, that is, $0 < |t - s_j|$. □

B.4 Coded profiles

Definition 3.5 presents a profile as a pair: a family F of ramp functions and a monotone functional λ on it. Constructively, the problem is that being handed an element of F "as a function" does not let one extract the witnesses of its behaviour that the proofs need.

Definition B.10 (Coded profile)**.** A *coded profile* on $[a, b]$ consists of: a type C of *codes* with an interpretation $\mathrm{ev} : C \times [a, b] \to \mathbb{R}$ and a subfamily $F \subseteq C$; boundedness, $0 \le \mathrm{ev}(f, x) \le 1$ for $f \in F$ and $a \le x \le b$; codes for the two constants, with $\mathrm{ev}(0, \cdot) \equiv 0$ and $\mathrm{ev}(1, \cdot) \equiv 1$; *separation*: given $a \le u$, $0 < v - u$ as data, and $v \le b$, one can *construct* a code $f \in F$ whose interpretation is 0 on $[a, u]$ and 1 on $[v, b]$; and a value functional $\lambda : C \to \mathbb{R}$, monotone along pointwise comparison of interpretations.

The decisive point is that the separation clause reads "one can construct"—it *returns* a code. If F were an extensional set of functions, obtaining an actual separating function from its mere existence would require a choice principle; making codes first-class objects and defining λ on codes removes that extraction from the outset. Where the source silently "takes a ramp function", the development holds a function that returns a code. The same shape—carry presentations, define functionals on presentations, never extract witnesses from extensional objects—is reusable at completions, at simple-function representations, and in the passage from measures to integrals.

B.5 The explicit exception sequence

Theorem B.11 (Explicit-sequence form of the smoothness theorem)**.** *For an integrable h and an interval (a, b), one constructs an* explicit sequence $T = (T_n)_{n \ge 0}$ *of presented reals such that every $t \in (a, b)$ apart from every T_n is a smooth level for the profile attached to h.*

Only in this form can Theorem B.4 be applied, with $(s_j) := T$. Two clarifications keep the claim exact. First, stating "the exceptions are countable" as "the exceptions are *this sequence*" is *not* a mathematical strengthening of the source: the source's proof of its Theorem 3.5 itself constructs the finite stagewise families $\{t_1^k, \dots, t_{n(k)}^k\}$ and their union T. The work of the formalization is flattening the proof-internal data into a first-class $\mathbb{N}$-indexed sequence, fixed as an interface that the avoidance theorem consumes directly. Second, the reason this does not collide with the no-go results is the confluence stated in Section 8, not the explicit-sequence form alone: over an abstract real-number object, a principle "from an explicit sequence, return a real apart from all its terms" would itself fall under the no-go; the avoidance construction of Appendix B.3 runs over presented-real order data, and that restriction is what carries the argument.

B.6 The truncation clause of Definition 1.1

The truncation demanded by clause (1) of the source's Definition 1.1 is $\min\{f, 1\} \in L$ *only*; the source's own remark recovers positive-integer truncation via $\min\{f, n\} = n \cdot \min\{n^{-1} f, 1\}$. The following four statements delimit what can and cannot be recovered.

Proposition B.12 (Positive constants)**.** *If $0 < \alpha$ is held as data, then clause (1) yields $\min\{f, \alpha\} \in L$: indeed $\min\{f, \alpha\} = \alpha \cdot \min\{\alpha^{-1} f, 1\}$, and the right-hand side lies in L by $\min\{\cdot, 1\}$ and closure under scalars. Constructing α^{-1} requires evidence that α is apart from 0—exactly "$0 < \alpha$ as data" in the sense of Definition B.2.*

Proposition B.13 ($\alpha = 0$)**.** *Clause (1) yields $\min\{f, 0\} \in L$: indeed $\min\{f, 0\} = \frac{1}{2}(f - |f|)$, and the right-hand side lies in L by $|\cdot|$ and linear combination. The verification goes through the lattice identity $\min\{a, b\} = \frac{1}{2}(a + b - |a - b|)$, valid on presented reals* without case analysis*, at $b = 0$; a case split on the sign of $f(x)$ is not decidable, so the identity is the constructively correct route.*

Remark B.14 (What $0 \le \alpha$ alone does not supply in the source route). Proposition B.12 consumes a positivity witness and Proposition B.13 exact zero; the split between the two cases is undecidable, so

no uniform construction from $\neg(\alpha < 0)$ alone is obtained here. Its impossibility is *not* proved (that would require a model separation); the claim is only that the source's route needs the witness, and that no other constructive route is evident. The source-derivable clause (Remark 2.2) is thus truncation at constants carrying a positivity witness, or at 0—the two kinds of constant the development in fact truncates at. The interface implements exactly this clause: truncation enters as a field only for constants carrying positivity data (`cutPos_mem`), truncation at 0 is the derived lemma of Proposition B.13 (`cutZero_mem`), and the witness type `CutConstWitnessC`, with a positive branch and a zero branch, carries the evidence to every use site.

Proposition B.15 (Negative constants fail in the non-compact canonical model)**.** *For $\alpha < 0$, closure under $\min\{\cdot, \alpha\}$ fails in the non-compact instances of the source's own canonical model. In the model $(X, C(X), \mu)$ of its Definition 7.1—X locally compact, $C(X)$ the continuous functions of compact support, μ a positive measure—one has $\min\{f, \alpha\} = \alpha \neq 0$ off the support of f, so for non-compact X the truncation has no compact support and leaves $C(X)$.* □

Consequently an interface demanding truncation at arbitrary constants and the source's Definition 1.1 would not be ordered by logical strength. With the witness-carrying clause (Remark B.14), Definition 2.1 is strictly weaker, at the clause level, than the encoded Definition 1.1; the present proposition is the design reason for the witness-carrying form.

Proposition B.16 (Positivity of I is derivable)**.** *Clause (2) implies: if $g \in L$ is nonnegative everywhere on its domain, then $I(g) \geq 0$.*

Proof. *First, Corollary 1.3 of the source at $k = 1$: if $I(g) > 0$ then $g(x) > 0$ at some point x of its domain. Apply clause (2) to the sequence $z_n := 0 \cdot g$ (each $z_n \in L$ with the domain of g, pointwise nonnegative, $\sum_n I(z_n) = 0 < I(g)$); it returns a point x with $\sum_n z_n(x) = 0 < g(x)$. Now suppose $I(g) < 0$. Then $I(-g) = -I(g) > 0$ and $-g \in L$, so $-g(x) > 0$ at some point, i.e. $g(x) < 0$, contradicting $g \geq 0$. Since $I(g) \geq 0$ abbreviates $\neg(I(g) < 0)$, this is a constructively valid conclusion.* □

Hence the positivity axiom (A4) of Definition 2.1 need not be carried as a field; the derivation is machine-checked in the artifact.

B.7 The two limits of the point-evaluation model

Fix a point x_0, let L be the partial functions defined at x_0, and let $I(f) := f(x_0)$. Clauses (1)–(3) are immediate—the constant 1 lies in L with $I(1) = 1$, so the model is normalized—and clause (4) reduces to two statements about the single real $c := f(x_0)$.

Theorem B.17. *For every $c \in \mathbb{R}$, with explicit moduli: (a) $\min\{c, n\} \to c$; indeed with $N := 2^{m(c)}$, where $m(c)$ is the exponent of an Archimedean bound for c, every $n \geq N$ gives $\min\{c, n\} \approx c$—the modulus is* constant*, and the convergence is exact from a finite stage on. (b) $\min\{|c|, 2^{-n}\} \to 0$; for gauge k, every $n \geq k + 2$ gives $|\min\{|c|, 2^{-n}\}| \leq 2^{-n} \leq 2^{-(k+2)} < 2^{-(k+1)}$.*

Proof. *(a) Choose a natural number $N \geq c$ (the Archimedean property, in the explicit form $N = 2^m$); for $n \geq N$ we have $c \leq N \leq n$, hence $\min\{c, n\} = c$. (b) $0 \leq \min\{|c|, 2^{-n}\} \leq 2^{-n}$, and $2^{-n} \leq 2^{-(k+2)} < 2^{-(k+1)}$ for $n \geq k + 2$.* □

Remark B.18 (Where the proposition/data gap does actual work)*.* For the Archimedean bound in (a), the development already possessed the *proposition* "there exists $N \in \mathbb{N}$ with $c \leq N$". A modulus, however, is data, and cannot be extracted from that proposition; the witness $2^{m(c)}$ inside the existing proof had to be written out as a function. "The existence was known, but a witness was needed" is a concrete work item that prose mathematics does not register; the same phenomenon reappears at the uniform complement estimate (Appendix B.10).

B.8 The A pair and the B pair: two complemented sets at one level, neither yielding the other

The source's Theorem 3.6 asserts that at a level t *two* complemented sets,

$$A = \big(\{h \geq t\},\ \{h < t\}\big), \qquad B = \big(\{h > t\},\ \{h \leq t\}\big),$$

are integrable with the same measure. Classically each pair consists of a set together with its ordinary complement, and at a continuity level the two positive components differ only on the null boundary $\{h = t\}$, so their characteristic functions agree almost everywhere; the two pairs are not obtained from one another by swapping components. Constructively they are not.

Definition B.19 (The four sets)**.** The value of an integrable h at a point x is given as data: the absolute convergence at x of a series representing h, together with the sum $s(x)$. On that basis,

$$A^1 := \{x : t \leq s(x)\}, \quad A^2 := \{x : s(x) < t\}, \quad B^1 := \{x : t < s(x)\}, \quad B^2 := \{x : s(x) \leq t\},$$

each set including the condition that the value exists.

That A is complemented—elements of A^1 and A^2 are distinct—follows because $t \leq s$ and $s < t$ are incompatible: $t \leq s$ *is* $\neg(s < t)$ by definition, so the incompatibility is definitional and requires no comparison of s with t. Similarly for B.

Remark B.20 (Why B is not a corollary of A)*.* Classically the two components of B are ordinary complements, as are the two components of A; but B is not obtained from A by swapping components. At a continuity level the difference between $\{h \geq t\}$ and $\{h > t\}$ is the null boundary $\{h = t\}$, and the integrability of B follows from that of A by that classical null-boundary argument. Constructively $\leq$ is the *negation* of $<$, and $\{h \geq t\}$ and $\{h > t\}$ are not each other's negations; passing between them is a double-negation elimination. The classical complement route is therefore closed; and on the data route, the public data of the A pair does not directly yield the data of the B pair either. The B pair was accordingly constructed independently. (Formal non-derivability—that no construction could exist—is not claimed.)

Remark B.21 (Why the mirror is not a copy: approximation from opposite sides)*.* The ramp used to approximate the characteristic functions is

$$\mathrm{ramp}_{u,v}(y) \ = \ \max\Big(\min\big((v-u)^{-1}(y-u),\ 1\big),\ 0\Big)$$

(0 for $y \leq u$, linear on $[u, v]$, 1 for $y \geq v$; the reciprocal $(v-u)^{-1}$ is constructed from the positivity evidence of $u < v$). The two pairs use it in opposite directions.

For the A pair, the *upper* end is fixed at $v = t$ and the lower end $u_n = t - \mathrm{gap}_n$ rises to t from below. The resulting ramps form a *decreasing* sequence whose limit realizes $\chi_{\{h \geq t\}}$, and the directly usable implication is

$$\mathrm{ramp}_{u_n,t}(y) < 1 \implies y < t.$$

For the B pair, the *lower* end is fixed at $u = t$ and the upper end $v_n = t + \mathrm{gap}_n$ descends to t from above. The ramps form an *increasing* sequence whose limit realizes $\chi_{\{h > t\}}$, and what is usable is the *positive* evidence

$$\mathrm{ramp}_{t,v_n}(y) > 0 \implies y > t,$$

not the implication used on the A side.

The two constructions thus approximate the characteristic function from opposite sides; the monotonicity of the ramp sequences is reversed, and the directly usable implications are different

statements. This is why carrying out the B pair is not a renaming exercise: the chain—about 55 lemmas in the artifact—is laid out a second time with the monotonicity inverted, while the generic components, the signed double series and shell enumeration of Appendix B.9, are reused unchanged. On either side, the *strong* (narrow) set is reached by the direct implication and the *weak* (wide) set is supplied as its complemented partner; the strong side is $\{h < t\}$ for A and $\{h > t\}$ for B, and that is where the two constructions trade places.

B.9 Signed double series and the shell enumeration

Countable additivity arguments use totals of doubly indexed families (A_{ij}) both as iterated sums and as sums along a single enumeration. Classically, absolute convergence licenses free rearrangement; constructively, one must *fix* an enumeration before asserting anything.

Definition B.22 (Shell enumeration)**.** Decompose each $m \in \mathbb{N}$ uniquely as $m = N^2 + t$ with $0 \le t \le 2N$, and set

$$\mathrm{cell}(m) := \begin{cases} (N,\, t) & (t \le N) \\ (t - N - 1,\, N) & (t > N). \end{cases}$$

Then $\mathrm{cell} : \mathbb{N} \to \mathbb{N} \times \mathbb{N}$ is a bijection tracing, for $N = 0, 1, 2, \ldots$, the L-shaped shells $\{(i, j) : \max(i, j) = N\}$.

Theorem B.23 (Signed double series along a fixed enumeration)**.** *Suppose given* as data*: (i) for each i, the absolute row sum $R_i := \sum_j |A_{ij}|$ with modulus; (ii) the sum $\sum_i R_i$; (iii) the flattened sum $\Sigma_{\mathrm{flat}} := \sum_k A_{\mathrm{cell}(k)}$. Then the iterated sum $\Sigma_{\mathrm{iter}} := \sum_i \big(\sum_j A_{ij}\big)$ can be constructed, and $\Sigma_{\mathrm{flat}} = \Sigma_{\mathrm{iter}}$.*

Proof shape. *Split $A_{ij} = A^+_{ij} - A^-_{ij}$ with $A^+ := \max(A, 0)$ and $A^- := -\min(A, 0)$. Since $0 \le A^{\pm}_{ij} \le |A_{ij}|$, the comparison test transfers the summability data of (i)–(iii) to both halves; for nonnegative families the shell sum and the iterated sum agree by monotonicity of partial sums and squeezing; subtracting the two nonnegative results returns the signed case.* □

Three features carry the constructive content: the conclusion is an equality *for one fixed bijection*, not a rearrangement license; absolute convergence enters as modulus-carrying data, not as a bare assertion; and the sign decomposition meshes with the comparison test because max and min are controlled by absolute values.

B.10 The uniform complement estimate as data

The statement is Proposition 5.5; this subsection records why its conclusion is data and how the set A is built.

Why data. Classically "$\forall \varepsilon\, \exists A, N, \delta$" would suffice. Here the sequel takes the set A in hand and performs the next construction on it, so an existence statement is not enough: the one-line "take such an A" becomes a function returning the triple (A, N, δ). This is the same phenomenon as Remark B.18.

How A is built. In each dyadic window $(2^{-(n+1)},\, 2^{-n})$, choose a level α_n at which the profile of the majorant H is smooth—this is where the explicit exception sequence (Appendix B.5) and the avoidance construction (Appendix B.3) are consumed. Apply the level-set theorem at each α_n to obtain integrable $A_n = (\{H \ge \alpha_n\}, \{H < \alpha_n\})$ with its measure. Integrability of H controls $\mu(A_n)$, and for n large the integral of H off A_n is small; the full-set bounds $e_n \le H$ transfer the estimate to the errors *uniformly in n*.

In order of application, the chain is: explicit exception sequence (Appendix B.5) → countable avoidance applied to it (Appendix B.3) → integrable level sets at apart thresholds (Appendix B.8) → dyadic level selection → uniform complement estimate (this subsection) → Theorem 4.1. If the first link were not constructive, everything downstream would degrade to existence statements.

B.11 Routine, known, and difficult: a classification

What was received, and what is routine:

Item	Status	Ground
The four clauses of Definition 1.1; Lemma 1.2; Corollary 1.3; the arcs of Theorems 3.5, 3.6, 4.15	known (source)	taken from Bishop–Cheng as stated
$\min\{f, n\} = n \cdot \min\{n^{-1}f, 1\}$	known (source)	the source's own remark after Definition 1.1
The *idea* of countable avoidance by nested intervals	known (Bishop, standard)	statement and overall strategy standard
$\min\{f, 0\} = \frac{1}{2}(f - \lvert f\rvert)$; derivability of the positivity of I	routine to known	elementary; that the positivity field is unnecessary is an observation of this development
Respect lemmas for Bishop equality; reading $\exists$ as data; convergence with moduli	formalization routine	not new mathematics, but obligatory

The difficulties:

Item	Status	Ground
Cut points fixed in advance at a margin (Appendix B.3)	difficulty	comparing against a separated pair is Bishop's step; what the formalization supplies is a margin fixed before the point is read, at the price of discarding the middle
Coded profiles (Appendix B.4)	difficulty, reusable	no witness extraction from extensional objects; the separation clause *returns* a code
Explicit exception sequence (Appendix B.5)	difficulty	"all but countably many" must become "away from this sequence" before avoidance applies
Mirror of the A pair and the B pair (Appendix B.8)	difficulty	$\leq$ is $\neg <$, so B is not a corollary of A; approximation from opposite sides reverses monotonicity and usable implications
Signed double series with shell enumeration (Appendix B.9)	difficulty, reusable	an equality for a fixed bijection instead of a rearrangement license; absolute convergence as modulus-carrying data
Uniform complement estimate as data (Appendix B.10)	difficulty	the triple (A, N, δ) must be returned, not asserted
One-point certificates and strong gauge extraction (Appendix B.12)	known (Bishop) + difficulty, reusable	the one-point diagonal condition and its tail equivalence are Definition 3 and Lemma 2 of *Foundations*, Chapter 2; the difficulty is executing the round trip, under the proposition/data distinction, without choice

The observations recorded by this development:

Item	Status	Ground
Analysis of the truncation clause (Appendix B.6)	observation	a positivity witness is needed; the source's canonical model refutes closure at negative constants; an arbitrary-constant clause would not be ordered against Definition 1.1
Resolution of the interface difference (the adapter)	result of this development	all fourteen fields follow from Definition 1.1 with no extra hypothesis, with clause (4) stated at the memoir's own gauge and the bridge to the working dyadic gauge machine-checked
The proposition/data gap surfacing as work (Remark B.18)	methodological observation	an existential proposition does not yield a modulus

The rows labelled "difficulty" are difficulties of the *formalization*, not claims of mathematical novelty. The constructions of Appendices B.3, B.4, and B.5 rewrite, into constructively executable form, operations that the source's proofs perform implicitly; none is a new theorem. What this formalization adds is that carrying them out determined *what is needed* to perform them—margins, codes, explicit sequences, positivity witnesses, one-point certificates. Those five items are what this appendix hopes to communicate to a mathematician.

B.12 One-point certificates and strong gauge extraction: a choice-free round trip between propositions and data

Section 8(i) said that the cotransitivity branch is returned as data from rational approximations; Section 9 said that every passage from propositions to data in the audited chain proceeds by explicit supply of data or by decidable search. This subsection writes out the decidable search. As Section 10.1 noted, unconditional extraction of data from an existential proposition is, in Lean, the territory of the choice axiom. Nevertheless, for the *strong order* on presented reals, one can pass between proposition and data without choice. This is a kind of statement for which classical published mathematics has no place—a conversion theorem between propositions and constructions—but it is not new to formalization, and the nearest instance lies in the library underneath the development compared with here: Coq's `ConstructiveReals` carries `CRltEpsilon` and `CRltForget` as fields, and its Cauchy-real instance discharges `CRltEpsilon` by decidable search over the rationals [31]. What follows is the same conversion, reached independently and built here from Bishop's own diagonal condition rather than posited as an interface field; type-theoretic literature has statements of the same kind as well (Appendix B.13).

The one-point condition itself is not an invention of this development. Definition 3 in Chapter 2 of Bishop's *Foundations defines* positivity of a real by exactly a one-point diagonal condition, $x_n > n^{-1}$ for some n, and its Lemma 2 supplies the equivalence with the tail form; the margin $2N^{-1}$ appearing there corresponds to the $2\varepsilon_n$ below, which is the dyadic-gauge version. What this subsection adds is the reading of that equivalence under the proposition/data distinction, and the verification that the round trip is executable, mechanically and without choice.

Definition B.24 (Tail positivity, in two forms)**.** Write $\varepsilon_k := 2^{-k}$. For a regular sequence $x = (x_n)$ (so $|x_n - x_m| \le \varepsilon_n + \varepsilon_m$), distinguish two forms of *tail positivity*: the *propositional form*, that there exist k and N such that $\varepsilon_k < x_n$ for all $n \ge N$; and the *data form*, the triple (k, N, proof) itself. The strong order $0 < x$ of Definition B.2 coincides by definition with tail positivity of x.

The propositional form contains a universal quantifier inside the existential—each instance is a decidable rational comparison, but there are infinitely many—so it cannot be searched as it stands.

Definition B.25 (Strong one-point certificate: the diagonal gauge)**.** For a regular sequence x put

$$W_x(n) \;:\Longleftrightarrow\; 2\varepsilon_n < x_n.$$

This is decidable by a single rational comparison. The point is that the threshold $2\varepsilon_n$ is tied to the gauge of the inspected index n itself: the certificate is *diagonal*.

Lemma B.26 (Self-propagation of the certificate)**.** *If $W_x(n)$ holds, then $\varepsilon_{n+1} < x_M$ for every $M \geq n+2$; hence $(k, N) := (n+1,\, n+2)$ is tail-positivity data.*

Proof. *For $M \geq n+2$ we have $\varepsilon_M \leq \varepsilon_{n+2}$. Regularity gives $x_M \geq x_n - (\varepsilon_n + \varepsilon_M)$, and the hypothesis gives $x_n > 2\varepsilon_n$, so*

$$x_M \;>\; 2\varepsilon_n - \varepsilon_n - \varepsilon_M \;=\; \varepsilon_n - \varepsilon_M \;\geq\; \varepsilon_n - \varepsilon_{n+2} \;=\; 3\varepsilon_{n+2} \;>\; \varepsilon_{n+1}. \qquad \square$$

The margin $2\varepsilon_n$ is engineered so that after regularity consumes its maximal budget $\varepsilon_n + \varepsilon_M$, one finer gauge ε_{n+1} still survives. It is the one-point version of the same idea as the σ-margin of Appendix B.3: *pay a margin in advance to make a comparison decidable.*

Lemma B.27 (Tails contain late certificates)**.** *If x has tail positivity in the propositional form, with witnesses k, N, then $\exists n\, W_x(n)$. Indeed $n_0 := N + k + 2$ gives $2\varepsilon_{n_0} \leq 2\varepsilon_{k+1} = \varepsilon_k < x_{n_0}$.* $\square$

This inference goes from a proposition to a proposition, so opening the existential inside it is legitimate—under the BHK reading and in Lean alike. What has changed is essential: an existential whose body contains a universal has been replaced by the existential of a *decidable one-point predicate.* Lemmas B.26 and B.27 are the dyadic-gauge version of Lemma 2 of *Foundations*, Chapter 2.

Proposition B.28 (Choice-free round trip)**.** *For a presented real x, the propositional form of $0 < x$ and the tail-positivity data are interconvertible without any choice principle.*

Proof. *Data to proposition is forgetting. For the converse: by Lemma B.27, pass from the propositional form to the existential of the decidable predicate W_x. Test $W_x(0), W_x(1), W_x(2), \ldots$ in order and return the first success—a* minimization search*. Termination is guaranteed by the existential; the search itself is well-founded recursion over a decidable predicate and requires no choice. (In Lean it is minimization through the accessibility predicate, whose elimination into data for this purpose is part of the calculus and, as Carneiro's axiomatization records, independent of the choice axiom; the axiom output of all constructions of this subsection lies within $\{$`propext`, `Quot.sound`$\}$ and is covered by the audit of Section 10.1.) The certificate so found is converted to tail data by Lemma B.26.* $\square$

One reading is worth recording. *Foundations* states that an element of $\mathbb{R}^+$ is properly a pair—a sequence together with a witness—and then declares that it will take "the position that the n was implicitly there all along", adopting the loose usage. Proposition B.28 is the machine-checked statement that this looseness is harmless without choice: the witness said to exist implicitly can always be recomputed, on the spot, whenever it is needed.

Remark B.29 (Consequences, the dual certificate, and the limitation). (i) *Cotransitivity as data.* Once evidence for $a < b$ is at hand—a witness index and its slack—comparison with any c is decided by a single rational comparison at that index; this is the mechanism behind the branch of Appendix B.3, and behind Section 8(i).

(ii) *Reciprocals.* The reciprocal of a presented real is a partial function taking positivity data as an argument. Abstract constructive real-number interfaces *posit* the reciprocal, with its positivity witness, as an axiomatic field—Coq's `ConstructiveReals` has `CRinv` taking an apartness witness, itself a sum of the two `Set`-valued order cases [31]—and each instance then discharges that field, the Cauchy-real instance by the same kind of decidable search used here; the presented side has no field to discharge, and computes the witness by the round trip before building the reciprocal. This is the smallest unit of the "abstract versus presented" difference of Section 9.1.

(iii) *The dual: an upper one-point certificate.* The same one-point-plus-regularity budget works upward: from the zeroth approximation alone,

$$|x_n| \;\le\; |x_0| + \varepsilon_n + \varepsilon_0 \;\le\; |x_0| + 2 \qquad (\text{all } n).$$

This is the faithful dyadic version of the standard bound K_x of *Foundations* (the least integer exceeding $|x_1| + 2$; the *Foundations* indexes its sequences from 1, so x_1 is the first approximation there, consistent with the gauge translation $x'_0 := x_{2^0} = x_1$), and it is why the Archimedean modulus in Appendix B.7 is constant. The positivity certificate from below and the boundedness certificate from above are two directions of one mechanism: regularity propagates one-point information to the whole sequence.

(iv) *Limitation.* The round trip exists only on the strong-order (apartness) side. The weak order $\le$ is *defined* as $\neg <$ and has no one-point certificate; this is the root of the fact that the weak sides of the pairs in Appendix B.8 are handled only through negation. Finally, none of this is a new theorem in the classical sense: in classical mathematics the proposition/data distinction collapses, and a conversion theorem between the two has no place in the traditional literature. What is worth reporting is that in a setting where the distinction does not collapse, the round trip holds without choice, and that this fact is machine-checked.

B.13 A dictionary with Booij's locators

The correspondence described here is conceptual; no locator type occurs in the artifact. Section and theorem numbers refer to Booij [25] in its arXiv version.

(i) The dictionary. Booij works in Martin-Löf type theory with propositional truncation $\|\cdot\|$, function extensionality, and propositional extensionality, and enforces the property/structure distinction down to the verbs ("there exists" against "has", "one can find", "one can construct"). Lean's `Prop`/Type split is the counterpart of the $\|\cdot\|$ discipline; his decidable propositions correspond to decidability data; and his search engine—Escardó's theorem as presented in his Theorem 3.5.1, that for a decidable predicate on $\mathbb{N}$ the truncated existence yields the untruncated witness pair, constructed as the *least* witness (his §3.4–3.6)—corresponds to the minimization search of Proposition B.28. The ambient foundations differ (his is compatible with univalence; Lean has definitional proof irrelevance with UIP, Remark 10.1), but Booij notes that univalence is not needed for this range, so the dictionary is sound across the foundations.

(ii) Locators and their normal form. A locator for a real x is $\ell : \Pi(q, r : \mathbb{Q}).\, q < r \to (q < x) + (x < r)$ (his Definition 3.1.1): locatedness with the disjunction replaced by an untruncated sum; one real may have many locators. His Lemma 3.4.2 shows a locator equivalent to a *decidable predicate* ("locates right") together with two propositional soundness clauses. That is the same normal form as the one-point certificate: Definition B.25 is a decidable predicate, and Lemmas B.26–B.27 are its soundness; structure of this kind *is* a decidable predicate paired with propositional soundness.

(iii) The same engine, asymmetric costs. His central technique is to reason about locator structure through propositions and to construct unique answers by bounded search (his §3.4–3.5),

obtaining rational bounds (Lemma 3.6.1) and Archimedean structure $x < y \to \Sigma(q : \mathbb{Q}).\, x < q < y$ (Lemma 3.7.2); Proposition B.28 is the same principle applied to positivity. The costs, however, are asymmetric: from a presentation, bounds can simply be read off (Remark B.29(iii): the first approximation and a constant modulus suffice), whereas a locator hides the sequence, so that even a bound needs a search over an enumeration of $\mathbb{Q}$; that a bound can be obtained from the locator alone at all is what Booij marks "perhaps surprisingly". This is the plainest exhibition of what carrying representations buys.

(iv) Where presented reals sit. His Theorem 3.9.7, for *mere* existence: that a locator exists, that a signed-digit representation exists, that a rational Cauchy sequence with limit x exists, and that x is a Cauchy real, are equivalent; the corresponding statement at the level of structure is his Theorem 3.9.5, that locators and signed-digit representations are interdefinable. Presented reals carry modulus-bearing rational sequences by definition, so through this dictionary every object of the present development lives on the locator-carrying side. Indeed a presented real carries a canonical locator on paper: given $q < r$, pick n with $2\varepsilon_n < r - q$ and compare x_n with the midpoint $(q + r)/2$; the regularity margin decides $(q < x) + (x < r)$. His Theorem 3.10.4 states that, *presupposing the property* of being a Dedekind cut, a single locator suffices for the remaining structure (boundedness and roundedness in structural form) to follow—the same architecture as Remark B.29, where one certificate plus regularity yields bounds, comparisons, and moduli.

(v) The same taboo, dual disciplines. His Lemmas 3.10.1 and 3.10.3: a global assignment of locators to *all* Dedekind reals defines, through the observation "does the locator of x locate right at $0 < 1$", a strongly nonconstant map $\mathbb{R}_{\mathbf{D}} \to \mathbf{2}$, and the constructive taboo WLPO follows. The same phenomenon has an appearance here: the least witness returned by the minimization search depends on the presentation and does *not* respect Bishop equality—two presentations of one real may return different least witnesses—and collapsing the witness to the Bishop-equality quotient corresponds exactly to truncating the locator to a property on his side. The two responses are dual. Booij keeps locators outside the definition of the real-number type, as structure carried alongside it, so that extensionality is preserved (his Theorem 3.10.5 and Corollary 3.9.8 analyse where the truncation may be placed: digit representations for all Dedekind reals are equivalent to the coincidence of the Cauchy and Dedekind reals, which is not constructively provable). The present development does not define its reals as the quotient—the carrier stays on the presentation side—so the collapsing operation never arises, and it pays instead the discipline of using extracted witnesses only inside Bishop-equality-invariant conclusions (the respect obligations of Section 10.4). These are dual answers, on the extensional side and on the presented side, to one and the same obstruction.

(vi) The strategy, at both ends. Booij obtains the exact intermediate value theorem—for which Bridges–Richman-style constructive analysis ordinarily consumes dependent choice—without choice principles, by carrying locators (his Theorem 4.3.5); and his Theorem 4.2.1 shows the Riemann integral preserves locators when a uniform-continuity modulus is carried as data, with locators for the endpoints and a structure lifting the function to locators also among the hypotheses. The present development does the corresponding thing at the other end, in the measure theory that Richman singled out as a principal obstacle (Section 8): the Bishop–Cheng profile route to dominated convergence, whose countable-avoidance step meets the choice obstruction discussed in Section 8 in unrestricted settings, is confined to {`propext`, `Quot.sound`} by presenting the reals and carrying witnesses. The two are instances, at the elementary-analysis end and at the measure-theory end, of one strategy: *replace invocations of choice by carried structure.* His open problem—whether pointwise continuous functions lifting to locators must carry structural continuity—does not arise in the present design, where continuity and convergence moduli are data from the start; that is a consequence of the difference in starting points, not a superiority of either.

References

[1] Errett Bishop. *Foundations of Constructive Analysis*. McGraw-Hill, New York, 1967.

[2] Errett Bishop and Henry Cheng. *Constructive Measure Theory*. Memoirs of the American Mathematical Society, no. 116, American Mathematical Society, Providence, 1972.

[3] Errett Bishop and Douglas Bridges. *Constructive Analysis*. Grundlehren der mathematischen Wissenschaften 279, Springer, 1985.

[4] Percy J. Daniell. A general form of integral. *Annals of Mathematics*, 2nd ser., 19(4):279–294, 1918.

[5] Yuen-Kwok Chan. *Foundations of Constructive Probability Theory*. Encyclopedia of Mathematics and its Applications, Cambridge University Press, 2021. Chapter 4, "Integration and measure".

[6] Radu Diaconescu. Axiom of choice and complementation. *Proceedings of the American Mathematical Society*, 51(1):176–178, 1975.

[7] Fred Richman. Meaning and information in constructive mathematics. *The American Mathematical Monthly*, 89(6):385–388, 1982.

[8] Douglas S. Bridges, Hajime Ishihara, Michael Rathjen, and Helmut Schwichtenberg (eds.). *Handbook of Constructive Mathematics*. Encyclopedia of Mathematics and its Applications 185, Cambridge University Press, 2023. DOI: 10.1017/9781009039888.

[9] Yuen-Kwok Chan. A leisurely random walk down the lane of a constructive theory of stochastic processes. In [8], pp. 333–356.

[10] Francesco Ciraulo. Subspaces in pointfree topology: towards a new approach to measure theory. In [8], pp. 426–444.

[11] Fred Richman. Countable choice. In [8], pp. 515–524.

[12] Maria Emilia Maietti and Giovanni Sambin. The Minimalist Foundation and Bishop's constructive mathematics. In [8], pp. 525–563.

[13] Robert S. Lubarsky. Inner and outer models for constructive set theories. In [8], pp. 584–635.

[14] Helmut Schwichtenberg. Computational aspects of Bishop's constructive mathematics. In [8], pp. 715–748.

[15] Kenji Miyamoto. Application of constructive analysis in exact real arithmetic. In [8], pp. 749–776.

[16] Mark Bickford. Efficient algorithms from proofs in constructive analysis. In [8], pp. 777–805.

[17] Ulrich Berger and Monika Seisenberger. On the computational content of choice principles. In [8], pp. 806–825.

[18] Fred Richman. The fundamental theorem of algebra: a constructive development without choice. *Pacific Journal of Mathematics*, 196(1):213–230, 2000.

[19] Douglas Bridges, Fred Richman, and Peter Schuster. A weak countable choice principle. *Proceedings of the American Mathematical Society*, 128(9):2749–2752, 2000.

[20] Fred Richman. Constructive mathematics without choice. In P. Schuster, U. Berger, and H. Osswald (eds.), *Reuniting the Antipodes—Constructive and Nonstandard Views of the Continuum*, pp. 199–205, Kluwer, 2001.

[21] Robert S. Lubarsky. On the Cauchy completeness of the constructive Cauchy reals. *Electronic Notes in Theoretical Computer Science*, 167:225–254, 2007. DOI: `10.1016/j.entcs.2006.09.012`.

[22] Bas Spitters. Constructive algebraic integration theory without choice. In T. Coquand, H. Lombardi, and M.-F. Roy (eds.), *Mathematics, Algorithms, Proofs*, Dagstuhl Seminar Proceedings 05021, pp. 1–13, 2006. DOI: `10.4230/DagSemProc.05021.9`.

[23] Thierry Coquand and Erik Palmgren. Metric Boolean algebras and constructive measure theory. *Archive for Mathematical Logic*, 41:687–704, 2002. DOI: `10.1007/s001530100123`.

[24] Iosif Petrakis and Max Zeuner. Pre-measure spaces and pre-integration spaces in predicative Bishop–Cheng measure theory. *Logical Methods in Computer Science*, 20(4):2:1–2:33, 2024. DOI: `10.46298/lmcs-20(4:2)2024`.

[25] Auke B. Booij. Extensional constructive real analysis via locators. *Mathematical Structures in Computer Science*, 31(1):64–88, 2021. DOI: `10.1017/S0960129520000171`. Also `arXiv:1805.06781`; the section and theorem numbers cited in this paper follow the arXiv version.

[26] Herman Geuvers and Milad Niqui. Constructive reals in Coq: axioms and categoricity. In *Types for Proofs and Programs (TYPES 2000)*, LNCS 2277, pp. 79–95, Springer, 2002. DOI: `10.1007/3-540-45842-5_6`.

[27] Vincent Séméria. Nombres réels dans Coq. In *Actes des 31es Journées Francophones des Langages Applicatifs (JFLA 2020)*, pp. 104–111, 2020. HAL record `hal-02427360`; archived paper PDF.

[28] Claudio Sacerdoti Coen and Enrico Tassi. A constructive and formal proof of Lebesgue's dominated convergence theorem in the interactive theorem prover Matita. *Journal of Formalized Reasoning*, 1(1):51–89, 2008. DOI: `10.6092/issn.1972-5787/1334`.

[29] Sylvie Boldo, François Clément, and Louise Leclerc. A Coq formalization of the Bochner integral. Inria Research Report RR-9456; `arXiv:2201.03242v2`, 2022.

[30] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE 28)*, LNCS 12699, pp. 625–635, Springer, 2021. DOI: `10.1007/978-3-030-79876-5_37`.

[31] The Rocq/Coq standard library. *Constructive real numbers*: `Reals.Abstract.ConstructiveReals` and `Reals.Cauchy.ConstructiveCauchyReals`. Version 8.20.1, 2025. The abstract interface carries `CRlt` (`Set`-valued), `CRltLinear`, `CRltEpsilon`, `CRltForget` and `CRinv` as fields; the Cauchy instance represents a real as a rational sequence with a modulus.

[32] Mario Carneiro. Lean4Lean: Verifying a typechecker for Lean, in Lean. In *31st International Conference on Types for Proofs and Programs (TYPES 2025)*, LIPIcs vol. 384, pages 2:1–2:23. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026. 10.4230/LIPIcs.TYPES.2025.2.

[33] Mario Carneiro. *The Type Theory of Lean*. Master's thesis, Carnegie Mellon University, 2019.

[34] Freek Wiedijk. The de Bruijn factor. Poster, Theorem Proving in Higher Order Logics (TPHOLs 2000), 2000. cs.ru.nl/~freek/factor.

[35] The mathlib Community. The Lean mathematical library. In *Certified Programs and Proofs (CPP 2020)*, pp. 367–381, ACM, 2020. DOI: 10.1145/3372885.3373824.

[36] The mathlib Community. Mathlib.MeasureTheory.Integral.DominatedConvergence. Source at the artifact's pinned mathlib revision c5ea00351c28e24afc9f0f84379aa41082b1188f; accessed 2026-07-09.

[37] Vincent Séméria and the CoRN contributors. Bishop–Cheng constructive measure theory declarations in CoRN. Audited CoRN repository snapshot, commit ada7c0b497ff15dd67cf7932c6f20e143a2aee2f. Audited source file CMTMeasurableFunctions.v. Profile and countable-avoidance source CMTprofile.v.

[38] Luís Cruz-Filipe, Herman Geuvers, and Freek Wiedijk. C-CoRN, the constructive Coq repository at Nijmegen. In *Mathematical Knowledge Management*, Lecture Notes in Computer Science 3119, pp. 88–103, 2004.

[39] Toshihisa Urahigashi. choicefree-bc-dct: a Lean 4 artifact for choice-free Bishop–Cheng dominated convergence. Version 0.7.2. Software deposit, Zenodo, 10.5281/zenodo.22309608. Concept DOI (resolves to the latest version): 10.5281/zenodo.21850965. Version DOIs of earlier deposits: v0.4.1 10.5281/zenodo.22137161, v0.4.0 10.5281/zenodo.21854936, v0.3.0 10.5281/zenodo.21850966.